MICROSOFT FABRIC GOVERNANCE

Fabric Atlas

Workspace intelligence and governance for Microsoft Fabric

Catalog | Lineage | Governance | Access | Operations

FABRIC NATIVE OPEN SOURCE METADATA ONLY

Product version 2.7.12. Example deployment evidence: FGI-MAIN, captured October–November 2026.

November 2026

Contents

1 Executive summary **3**
1.1 What Atlas provides . . . 3

2 Why a workspace atlas is useful **4**

3 Example workspace state **5**

4 Workspace orientation **5**
4.1 Governance Radar Overview . . . 5
4.2 Item Catalog . . . 6
4.3 Workspace-wide search . . . 7

5 Internal asset inventory **8**
5.1 Asset Catalog . . . 8
5.2 Measure metadata and provenance . . . 9

6 Lineage and impact analysis **10**
6.1 Trusted Atlas graph . . . 10
6.2 Item Relations Preview . . . 11
6.3 Evidence desk . . . 12
6.4 Semantic model X-Ray . . . 13
6.5 Object lineage . . . 14
6.6 Exportable impact reports . . . 15

7 Governance and policy evidence **18**
7.1 Findings . . . 18
7.2 Change Center . . . 19
7.3 Metadata coverage . . . 20
7.4 Posture targets . . . 21
7.5 Policies & AI . . . 23

8 Governance extensions in 2.6 **23**
8.1 Domains and catalog discovery . . . 23
8.2 Explainable findings . . . 24
8.3 Tenant drift and Fabric policies . . . 25
8.4 Notifications and AI assistance . . . 25

9 What's new in 2.7 **25**
9.1 Atlas Scout . . . 25
9.2 External lineage (OpenLineage) . . . 28
9.3 Sync Brief . . . 29
9.4 Notebook object lineage . . . 29

10 Access review and departure planning **29**
10.1 Review matrix . . . 29
10.2 Principal-centred review . . . 30
10.3 Read-only What-if . . . 31

10.4 Departure packs . . . 32

11 Operations and workspace context **33**
11.1 Jobs & health . . . 33
11.2 Multi-workspace scope . . . 34
11.3 Synchronization . . . 35
11.4 Configuration . . . 36
11.5 Team notes . . . 37

12 Architecture and synchronization **38**
12.1 Collector ownership . . . 39
12.2 Why one synchronization can take several minutes . . . 40
12.3 Immutable snapshots . . . 40
12.4 Retention and writer trust . . . 40
12.5 Persisted entities . . . 41

13 Security and data boundaries **41**
13.1 Data Atlas stores . . . 41
13.2 Data Atlas excludes . . . 41
13.3 Authentication and authorization . . . 42
13.4 Failure behaviour . . . 42

14 Deployment and operating model **43**
14.1 Prerequisites . . . 43
14.2 Deployment flow . . . 43
14.3 First sync and upgrades . . . 43
14.4 Routine operation . . . 43

15 Fabric item coverage **44**

16 Read-only Atlas MCP **45**

17 Open source implementation **45**

18 Current limits **46**

19 Conclusion **47**

20 Appendix A: figure index **48**

21 Appendix B: glossary **49**

22 References **49**

This paper documents Fabric Atlas 2.7.12. FGI-MAIN is used as the real deployment example. Most figures were captured on 28 October 2026 with release 2.6.4; the Atlas Scout and external lineage figures were captured with release 2.7. Counts and findings describe that workspace at that point in time. They are not product defaults, limits or benchmarks.

1 Executive summary

Microsoft Fabric brings data engineering, analytics, real-time processing and business intelligence into one service. The workspace is where teams build and operate that estate, but the metadata needed to understand it is spread across item pages, REST APIs, the Power BI scanner, run history, permissions and definition files.

Fabric Atlas collects that metadata into a Fabric App. It indexes selected workspaces without copying business data. Users can browse Fabric items, inspect internal objects, trace lineage, review effective access, compare validated snapshots and keep operational notes beside the catalog.

Version 2.0 changes how the metadata is collected. Typed Rayfin Functions now own core inventory, definitions, Item Relations, SQL metadata, KQL structure, source provenance and access-policy evidence. A smaller Python User Data Function remains only for documented platform gaps: the Power BI admin scanner, PBIR-Legacy report pages, Kusto live metadata and explicit rollback.

Version 2.6 aligns Atlas with the unified governance model of Fabric and the OneLake catalog. It adds domain posture, catalog facets, explainable finding priority, tenant-setting drift, OneLake data access roles, external-share evidence, incremental synchronization, optional Teams notifications and reviewed AI description suggestions. Each new signal follows the same rule: evidence that cannot be collected stays Unknown.

Version 2.7 adds Atlas Scout, a plain-language governance chat over a published Fabric Data Agent that reads the Atlas SQL database. It also adds OpenLineage import and export for lineage outside Fabric, a Sync Brief that summarizes the latest synchronization, and notebook object lineage built from direct literal table references.

The browser coordinates those collectors, isolates item failures, validates every response and publishes the workspace manifest last. Each selected workspace has an independent snapshot. A failed refresh in one workspace cannot replace another workspace's catalog.

Atlas distinguishes observed facts from derived context:

- Atlas snapshot lineage is the trusted source for stored impact analysis.
- Item Relations Preview is an optional Beta view of raw API relationships.
- DAX dependencies are resolved only when a measure or column reference matches one synchronized object.
- Access What-if removes recorded grant paths locally. It never changes Fabric permissions and never claims to model unavailable restriction layers.
- Missing metadata remains unknown, unsupported, excluded or not applicable. It is not silently treated as healthy or compliant.

1.1 What Atlas provides

Area	Practical outcome
Workspace scope	A shared, administrator-selected list of workspaces with independent snapshots and active-workspace switching

Area	Practical outcome
Catalog	One searchable inventory of Fabric items with type, ownership, health, labels, tags, domain facets, My items scope and Request access links
Asset inventory	Relational, semantic, KQL, ontology, graph and Data Agent objects grouped under their parent item
Search	One command palette for items, objects, people, jobs and notes, with optional OneLake Catalog Preview integration
Lineage	Stable source-to-consumer graphs, object expansion, evidence review and focused impact paths
DAX X-Ray	Direct and transitive measure-to-column and measure-to-measure dependencies
Governance	Findings with priority and Why panel, domain posture, tenant-setting drift, changes, history, coverage, posture targets and policy context
Access review	Effective recorded access, principal and role filters, OneLake data access roles, external shares, local What-if scenarios and departure packs
Operations	Recent jobs, observed failures, inferred downstream reach, incremental sync, snapshot export and optional Teams alerts
Collaboration	Shared append-only notes attached to a workspace or Fabric item
MCP	Optional local read-only tools over validated Atlas snapshots

2 Why a workspace atlas is useful

Fabric already exposes extensive metadata. The difficulty is that a reviewer must move between several experiences to answer one operational question.

Consider a semantic model change. A reviewer may need to identify:

- the Lakehouse or Warehouse that supplies the model;
- notebooks and pipelines that update the source;
- reports and Data Agents that consume the model;
- measures that reference a changed column;
- people who can edit the model;
- the latest refresh status;
- policy or sensitivity evidence that Fabric exposed;
- workspaces at the other end of a reported relationship.

Each fact may exist, but not in one place and not always at the same point in time. Atlas treats the selected workspace scope as a versioned metadata graph. Catalog, lineage, access, governance and operations are different views of the same validated snapshots.

The snapshot model supports questions that a current-state list cannot answer:

- What changed between two accepted snapshots?
- Did a change introduce a new priority risk?
- Which consumers are reachable from this item?
- Which measures use this selected model column?
- Is an access path inherited, direct or mixed?
- Which evidence was unavailable rather than negative?
- Did another workspace retain its valid snapshot when this refresh failed?

Atlas does not replace Fabric administration, deployment or monitoring tools. It is a metadata reading and review layer. Links return users to Fabric when a portal-only control is required.

3 Example workspace state

The FGI-MAIN manifest used by the screenshots reports:

Observed value	Current evidence
Fabric items	90
Internal objects	3,069
Atlas lineage links	82
Principals	3
Stored access grants	252
Evaluated principal-item pairs	270
Recent job runs	14
Failed recent job runs	6
Configuration entries	1,889
Shared team notes	0

FGI, FGI-MAIN, FGI-ORACLE and MSFlix are selected in the shared Atlas scope. FGI-MAIN is the active workspace in the screenshots. The other workspaces remain separately addressable and are loaded only when a user switches to them.

4 Workspace orientation

4.1 Governance Radar Overview

The Overview is designed for orientation. The Governance Radar is the main visual, with workspace health and priority signals beside it. The current snapshot places four of six posture pillars at or above their 70 percent targets.

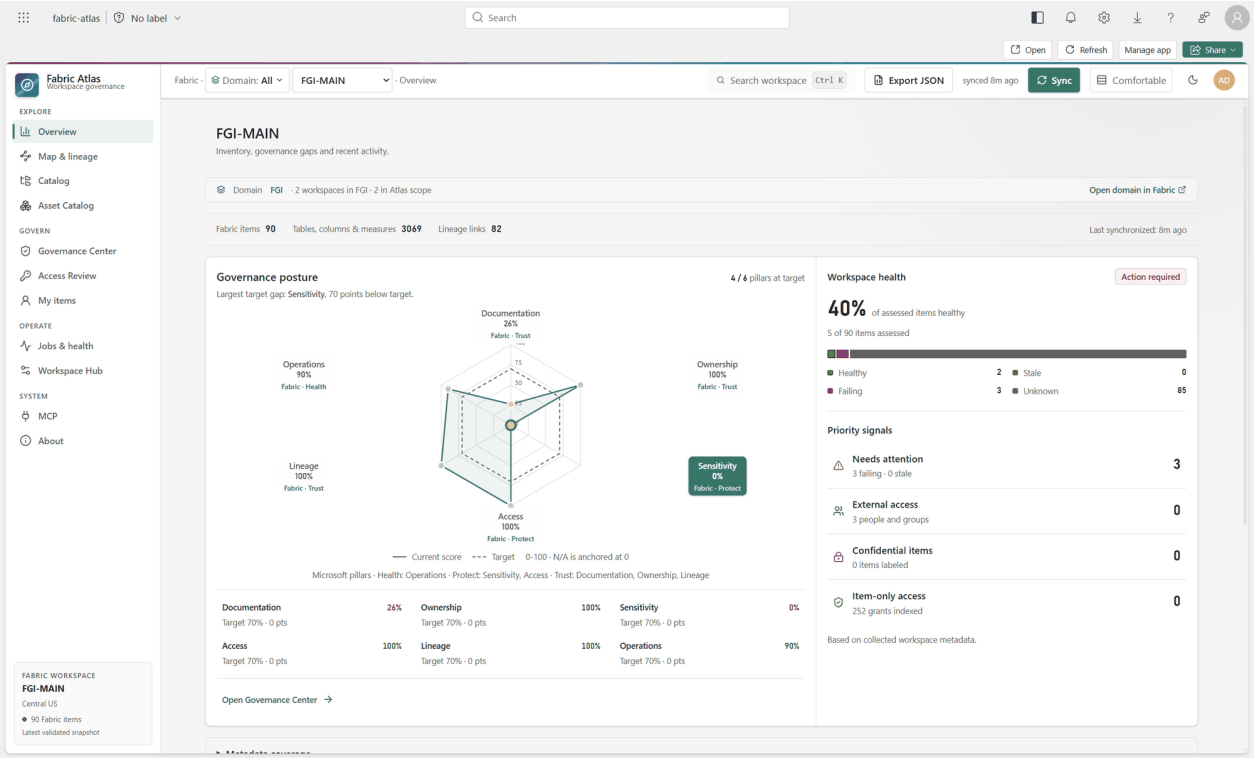


Figure 1: Governance Radar Overview for FGI-MAIN.

The screenshot reports:

- 90 Fabric items and 3,069 tables, columns and measures;
- 82 lineage links;
- 40 percent health across the five items with assessed health evidence;
- three failing items;
- no detected external or confidential item signal in the current workspace;
- 252 indexed access grants.

The Overview also shows the Fabric domain of the workspace and the largest posture gap, Sensitivity, 70 points below its target.

Unknown health does not count as failure. Eighty-five items have no assessed health status and remain visibly separate from healthy, failing and stale states.

4.2 Item Catalog

The Catalog keeps every item returned by Fabric visible, including types that do not yet have a dedicated deep collector. Users can switch between cards and a sortable table, search by name or tag and filter by item family.

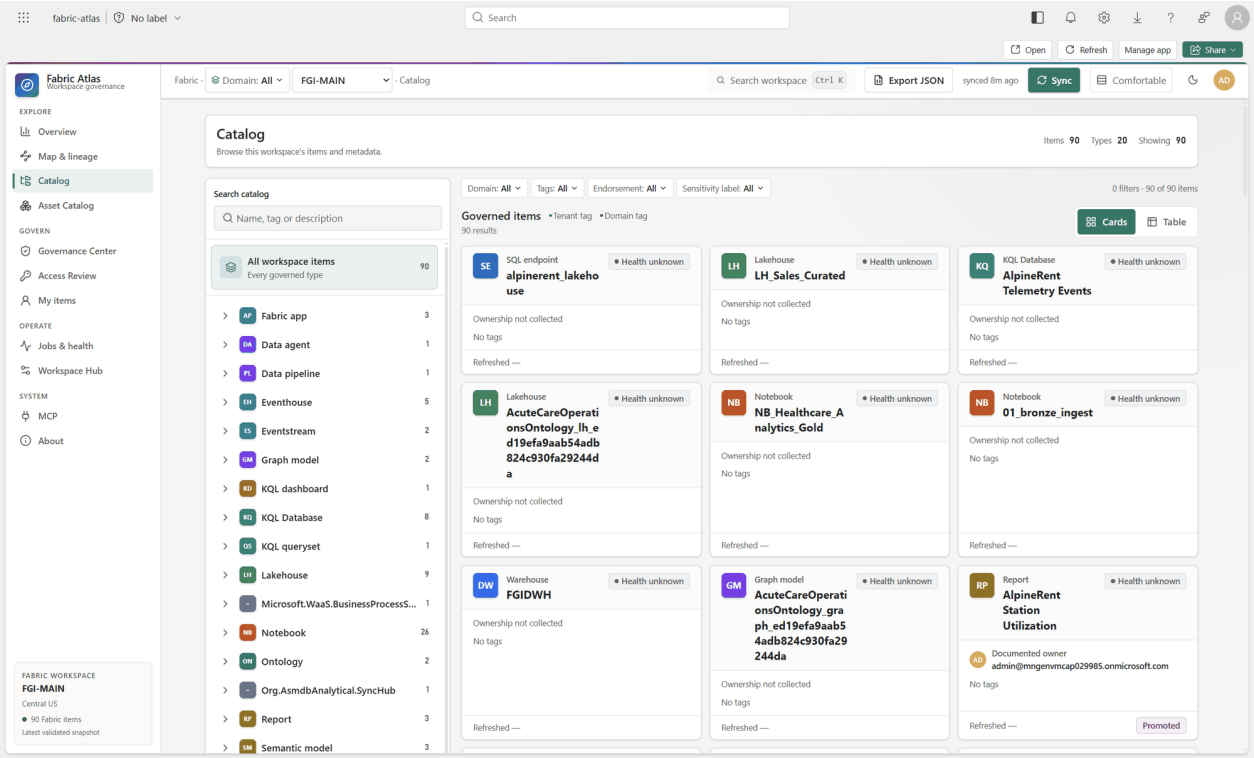


Figure 2: Fabric item Catalog for the active workspace.

Groups remain collapsed by default unless the user searches. This keeps a large workspace readable while preserving access to every item.

4.3 Workspace-wide search

`Ctrl+K` opens a command palette over the active snapshot. The search index covers workspaces, items, tables, columns, measures, principals, jobs, configuration and notes.

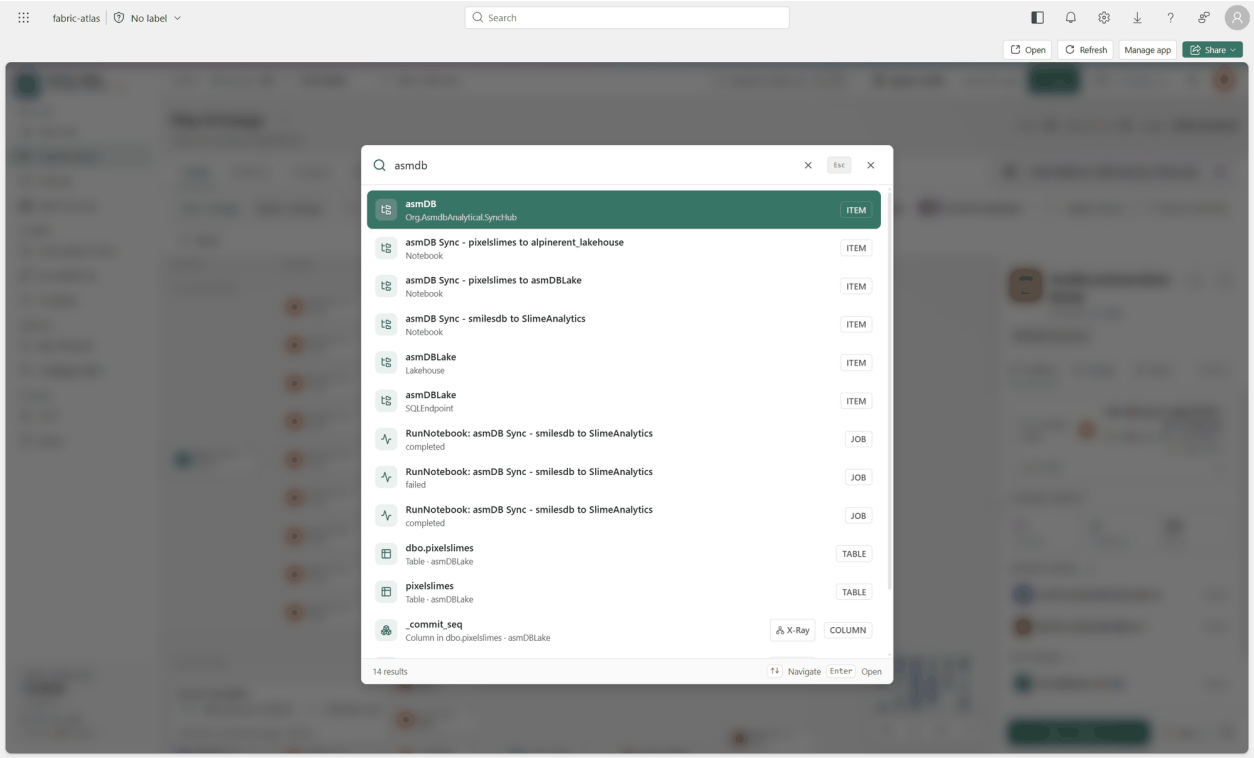


Figure 3: A real search for asmdb across items, jobs, tables and columns.

The `asmdb` search returns the Workload Hub item, related notebooks, `asmDBLake`, SQL endpoint metadata, completed and failed notebook runs and synchronized schema objects. Search results navigate back into the matching Atlas view.

OneLake Catalog Preview can extend the same dialog when the deployment enables the feature and the API is available. Results outside the selected Atlas scope open in Fabric instead of being copied into the snapshot.

5 Internal asset inventory

5.1 Asset Catalog

The Asset Catalog moves below the item boundary. Items and objects are sorted alphabetically. Filters separate tables, views, columns, measures and the specialized object kinds collected from KQL, SQL, ontology, graph and Data Agent definitions.

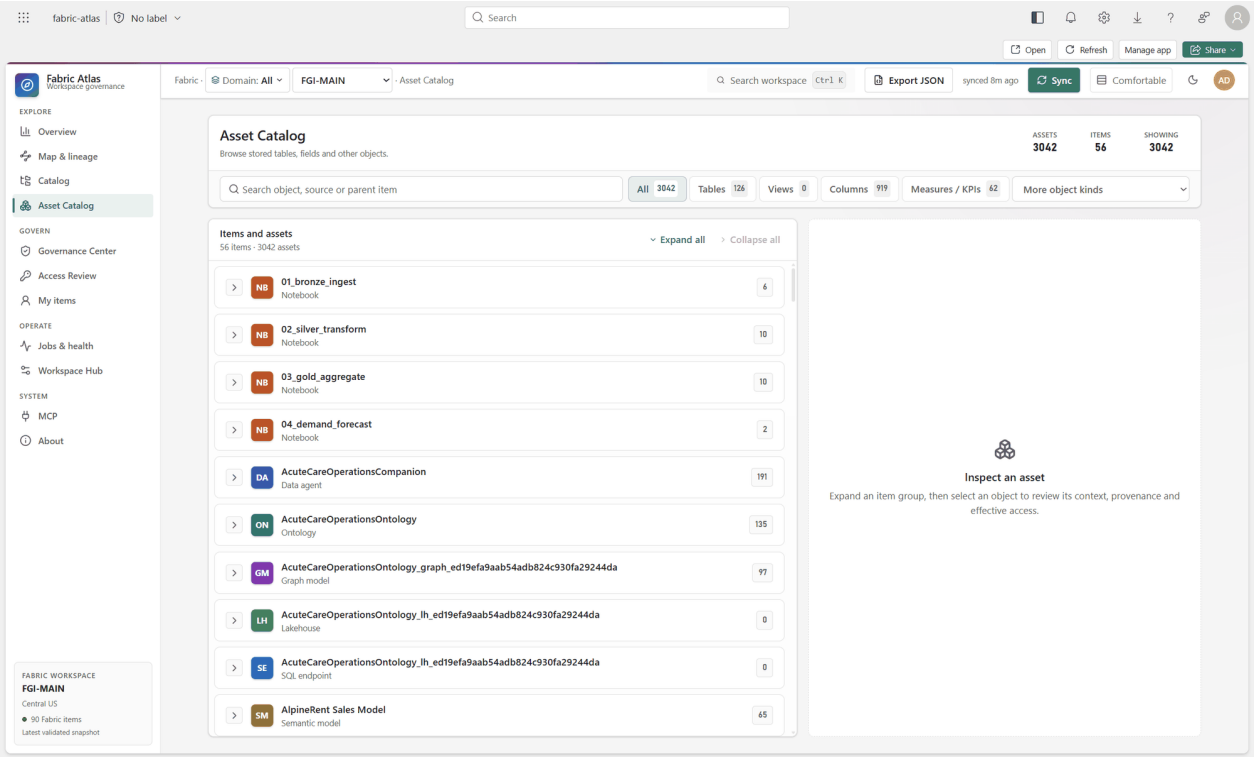


Figure 4: Alphabetized Asset Catalog with all synchronized object kinds.

The captured catalog contains 3,042 objects across 56 Fabric items. It includes generic model tables and columns plus SQL-specific, KQL-specific and definition objects. Items that support schema collection remain visible even when Fabric returns no objects.

5.2 Measure metadata and provenance

Measure expressions are retained when Fabric exposes them as semantic model metadata. Atlas strips comments and string literals before resolving qualified columns and measures against the synchronized schema.

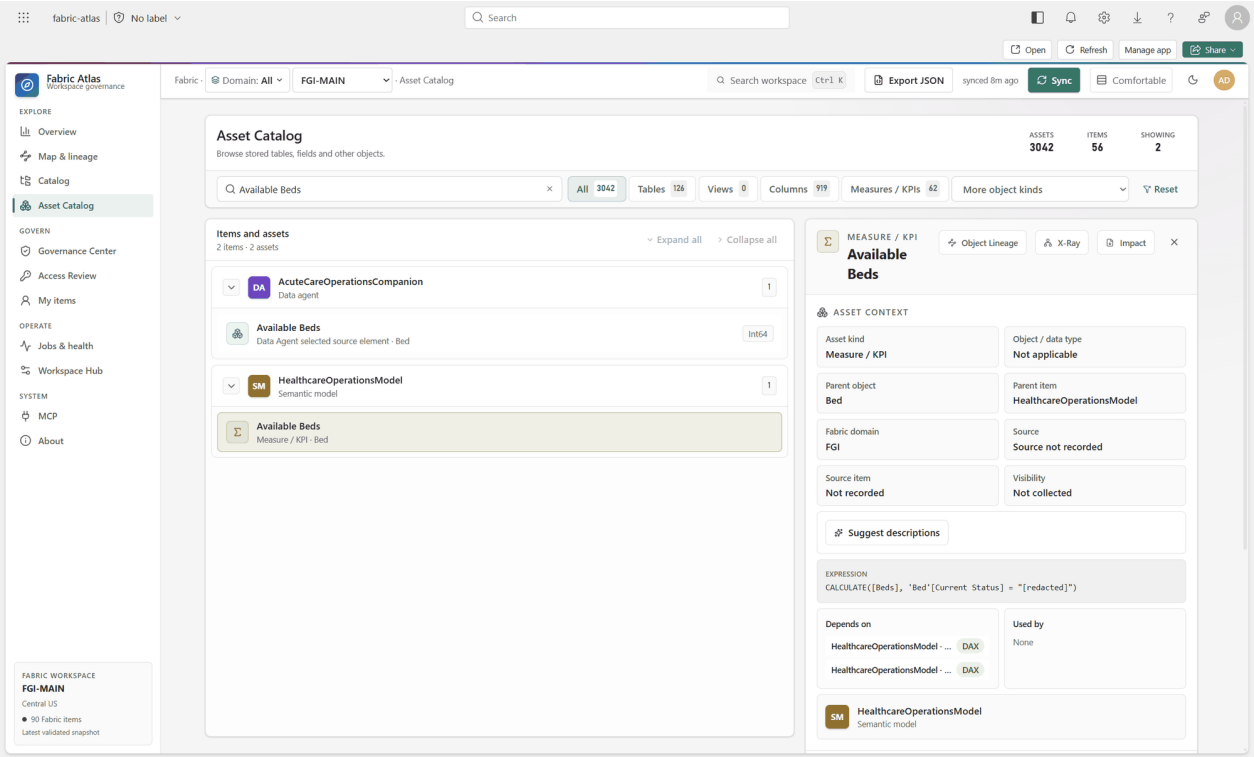


Figure 5: Available Beds with its DAX expression and dependency evidence.

`Available Beds` depends on the `Beds` measure and `Bed[Current Status]`. Atlas also exposes the unique upstream Lakehouse object when item lineage and schema identity support that hop. Same-model references are labelled DAX. Cross-item source hops remain labelled inferred.

Ambiguous references are omitted. Atlas does not create object lineage from name similarity alone.

6 Lineage and impact analysis

6.1 Trusted Atlas graph

The Atlas graph normalizes stored relationships from source to consumer. Its initial layout is staged from orchestration through storage, endpoints, models and consumers. The computed layout contains no backward edges in the initial view.

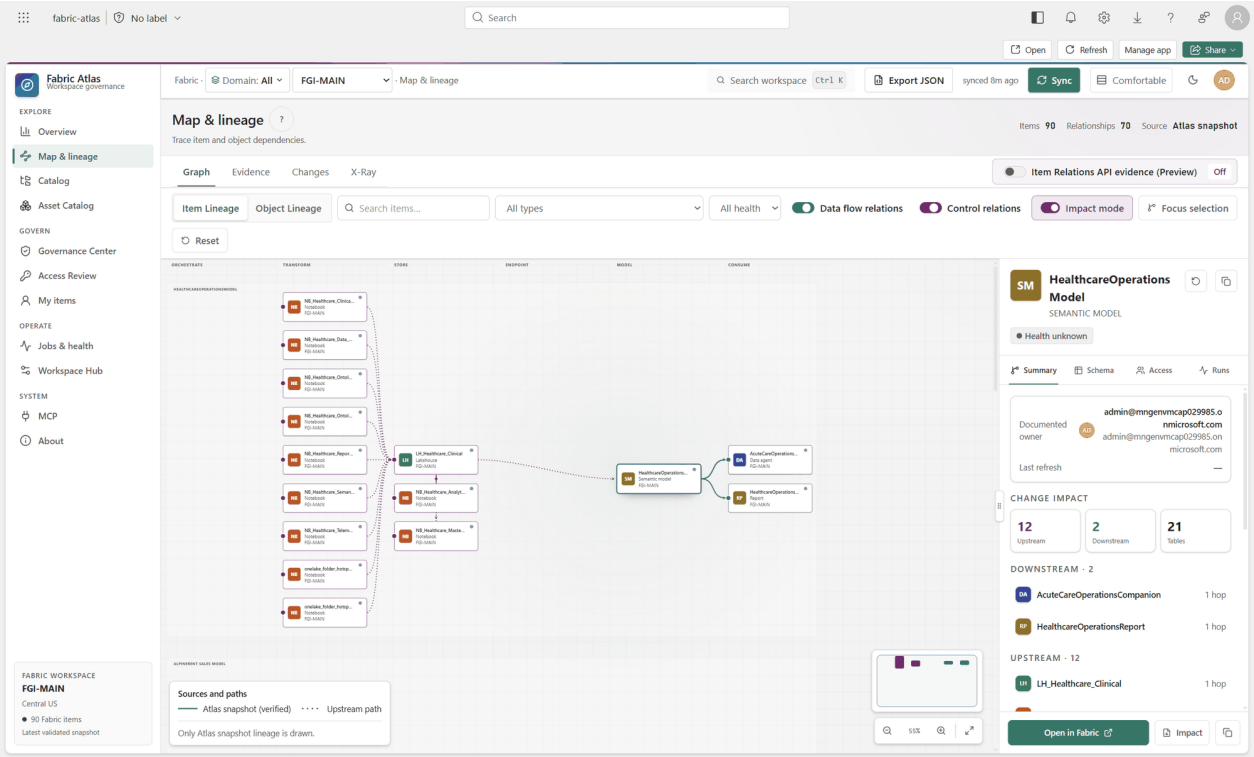


Figure 6: HealthcareOperationsModel in focused Atlas impact mode.

Impact mode removes unrelated components. The selected `HealthcareOperationsModel` keeps its upstream Lakehouse, the notebooks that load it two hops away, and its downstream report and Data Agent in view. The inspector reports the complete upstream and downstream path, not only the lines visible in the current viewport.

Users can move Atlas nodes freely. Reset clears selection, focus, impact, dragging and Preview expansion, then restores the computed layout.

6.2 Item Relations Preview

Item Relations Preview is a separate graph source. Enabling it replaces Atlas relationships rather than combining two sources into one graph.

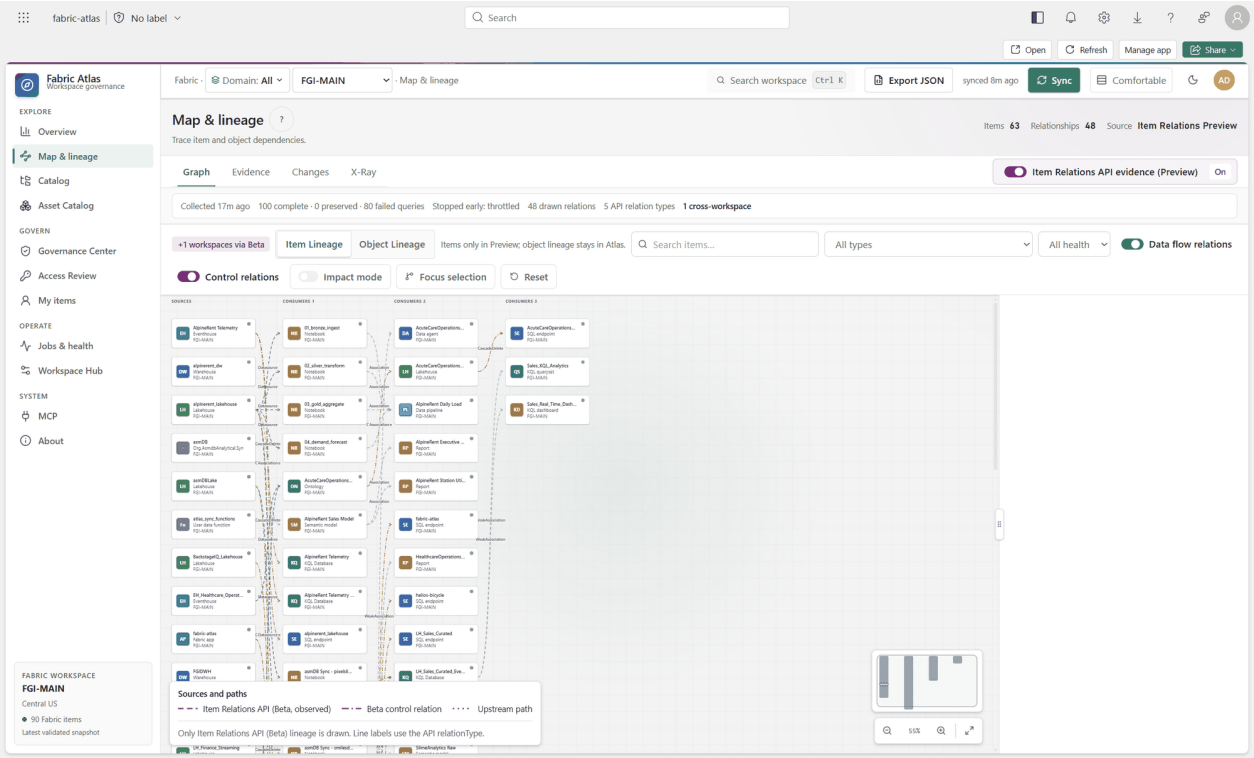


Figure 7: Item Relations Preview with raw relation labels.

Preview keeps the raw Fabric API `relationType` on each line. The captured evidence contains 48 drawn relationships across 63 items, five relation types and one cross-workspace relationship. Observed types include:

- `Association`;
- `CascadeDelete`;
- `Datasource`;
- `Shortcut`;
- `WeakAssociation`.

The status row also reports incomplete or throttled queries. In this capture, 100 item queries completed, 80 failed and the collection stopped early after throttling. Atlas preserves that boundary instead of presenting the result as a complete authoritative graph.

6.3 Evidence desk

The Evidence tab lists the relationships behind the current Preview graph. It shows source, target, raw relation type, workspace boundary and observation time.

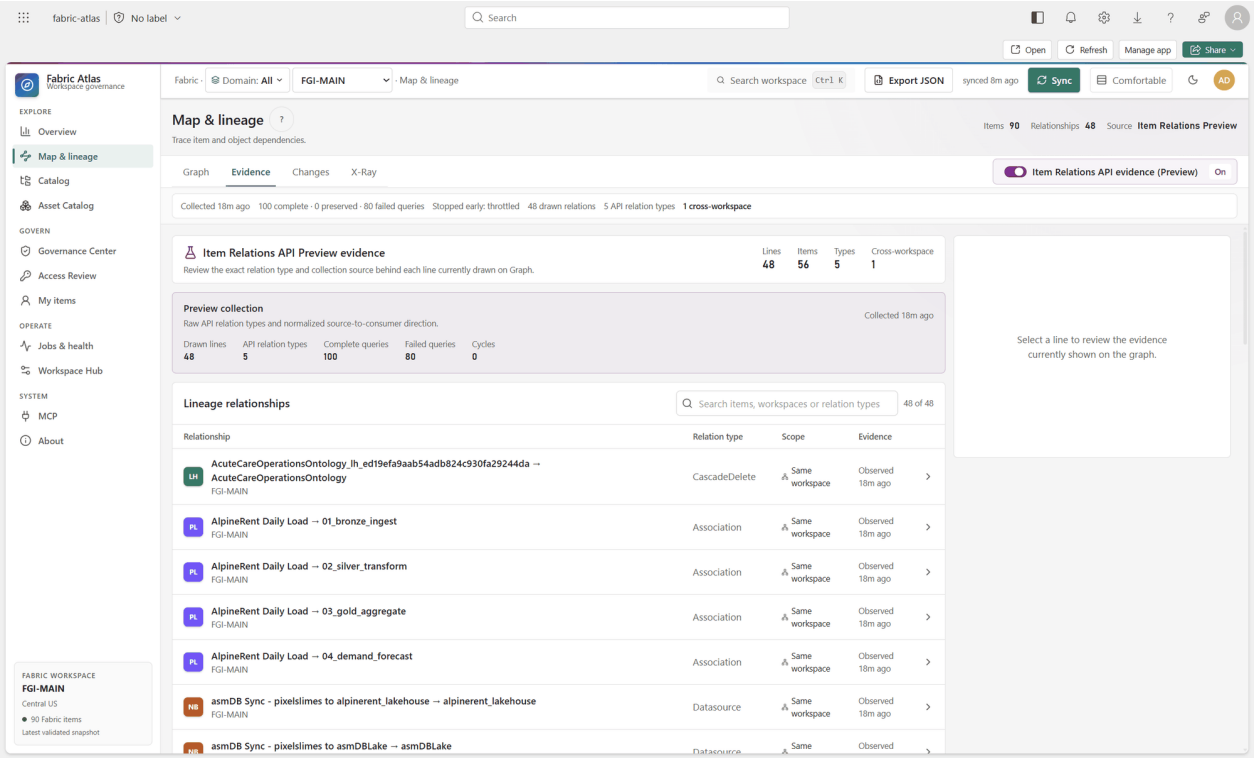


Figure 8: Evidence desk for Item Relations Preview.

The summary separates drawn lines, item count, relation types and cross-workspace evidence. A reviewer can search by item, workspace or relation type without returning to the graph.

6.4 Semantic model X-Ray

X-Ray reads the resolved DAX dependency index for one semantic model. It supports measure and column search, Depends on or Used by direction, and direct or transitive depth.

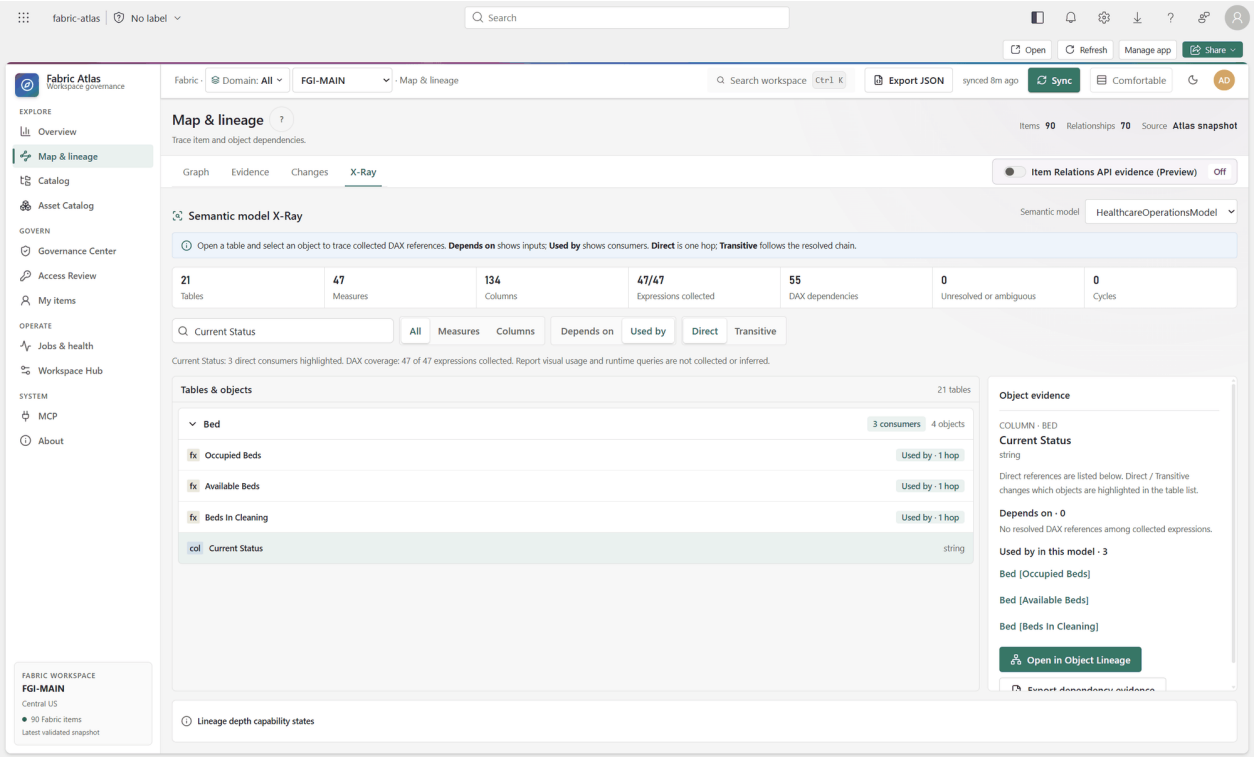


Figure 9: HealthcareOperationsModel X-Ray for Bed Current Status.

The selected model contains 21 tables, 47 measures, 134 columns and 55 resolved DAX dependencies. No unresolved or cyclic dependency is reported in this snapshot. Selecting `Bed[Current Status]` highlights three direct consumers: `Occupied Beds`, `Available Beds` and `Beds In Cleaning`.

X-Ray does not infer report visual usage or runtime query behaviour. It shows the dependency evidence that was collected from model metadata.

6.5 Object lineage

Object mode expands synchronized tables, columns and measures while retaining the surrounding item graph.

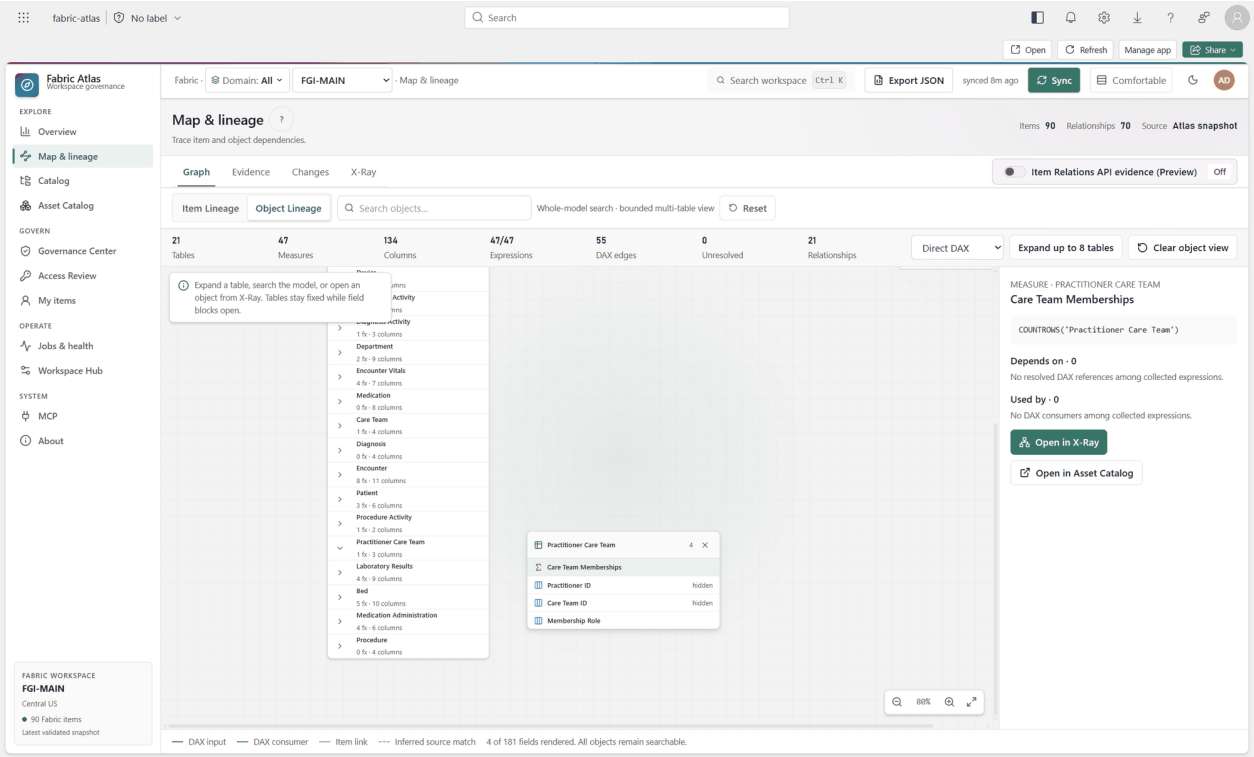


Figure 10: Object lineage from Practitioner Care Team to model fields and consumers.

Connected items use collapsible summary nodes. Selecting an object in another item switches the active context and rebuilds the graph. Users can pan the canvas, move boxes and expand or collapse all item groups.

The boundary remains explicit. Atlas can prove that a report consumes a semantic model without claiming that a particular visual uses a selected field.

6.6 Exportable impact reports

An item impact report combines upstream, downstream and relationship evidence from the trusted snapshot.

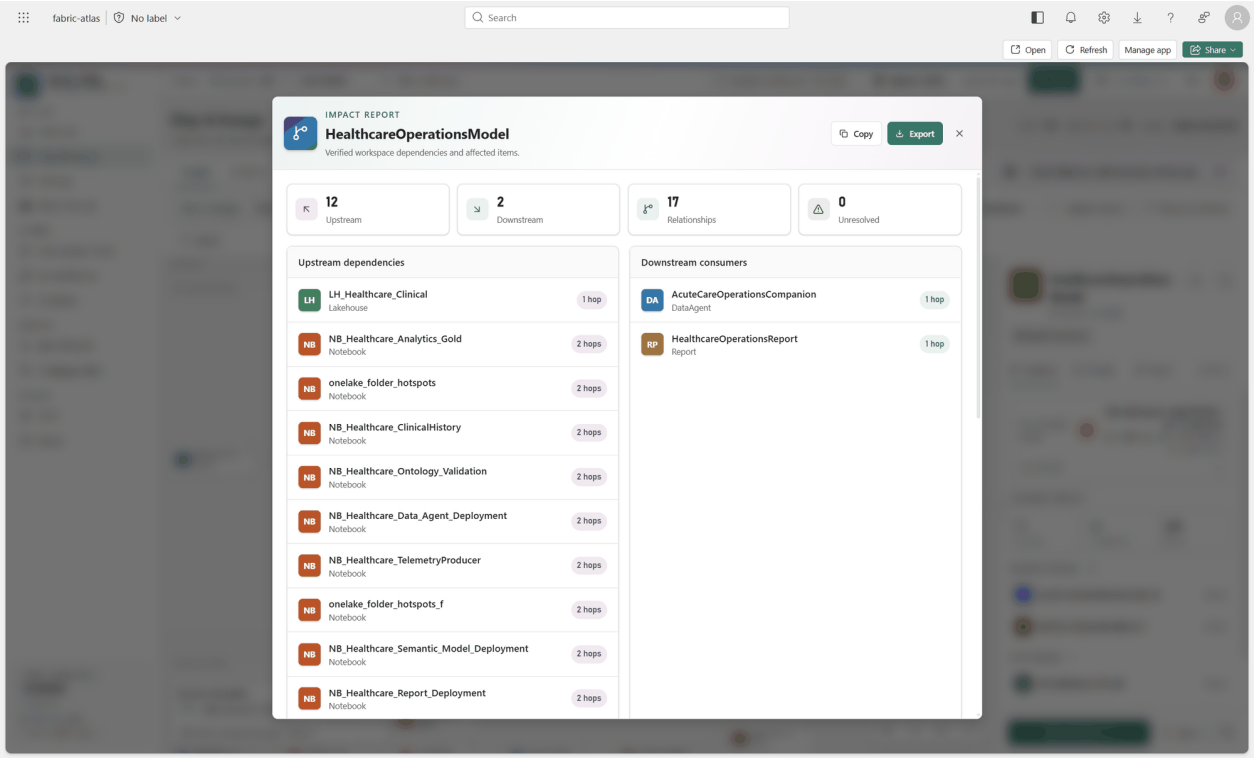


Figure 11: Semantic model impact report for HealthcareOperationsModel.

At object level, the Asset Catalog exposes reverse DAX consumers for a selected column.

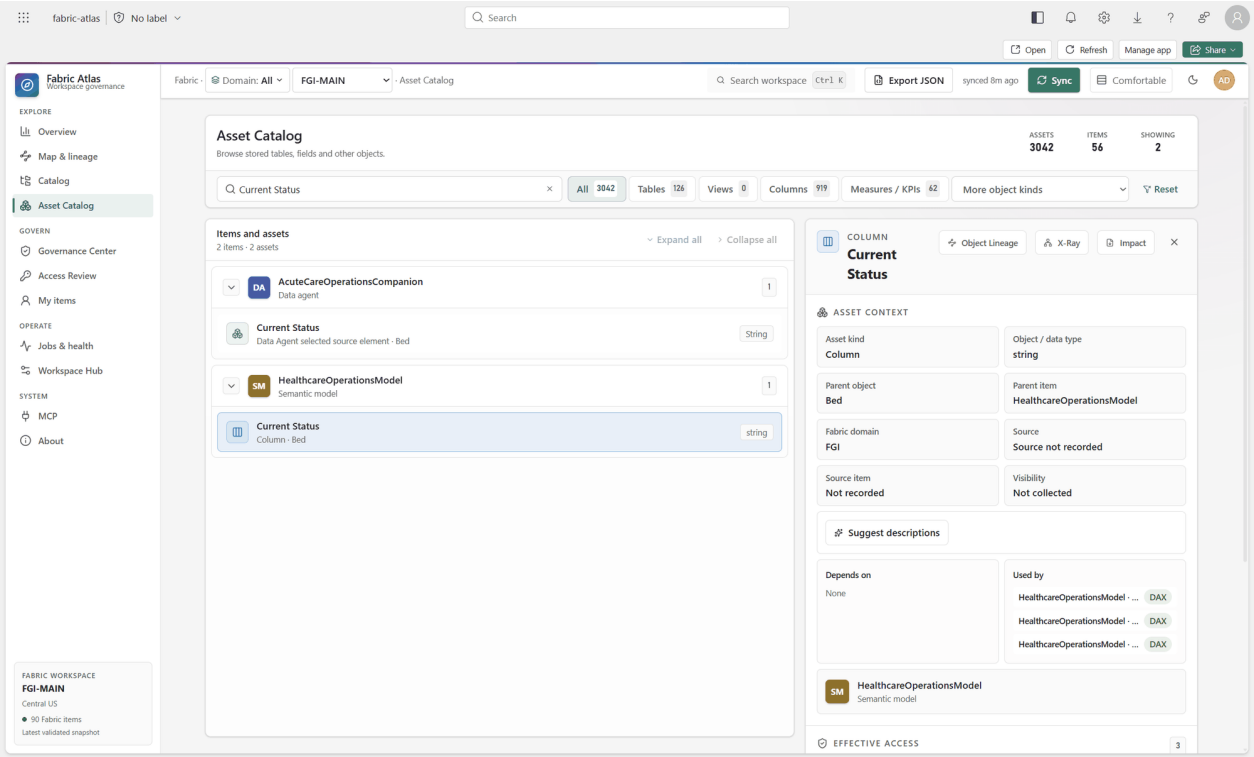


Figure 12: Bed Current Status with three verified DAX consumers.

The object impact dialog can export the same evidence as Markdown. It keeps verified DAX dependencies separate from inferred cross-item source hops and Data Agent selected-element evidence.

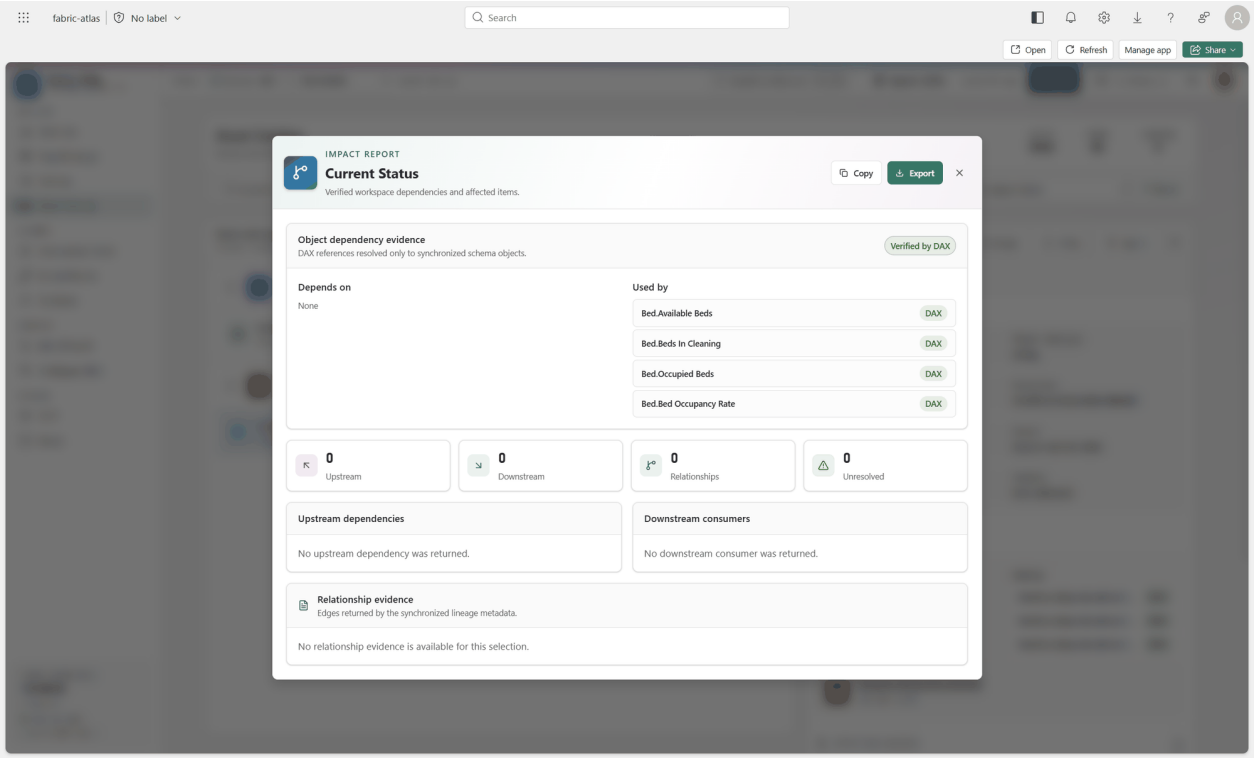


Figure 13: Exportable object impact report.

This distinction prevents an overclaim. Atlas can identify reports that depend on a semantic model and measures that depend on a model field. It cannot currently identify the exact report visual that uses that field.

7 Governance and policy evidence

7.1 Findings

Governance findings are reproducible checks over the current validated snapshot. They cover documentation, sensitivity, access, lineage, operations and metadata coverage.

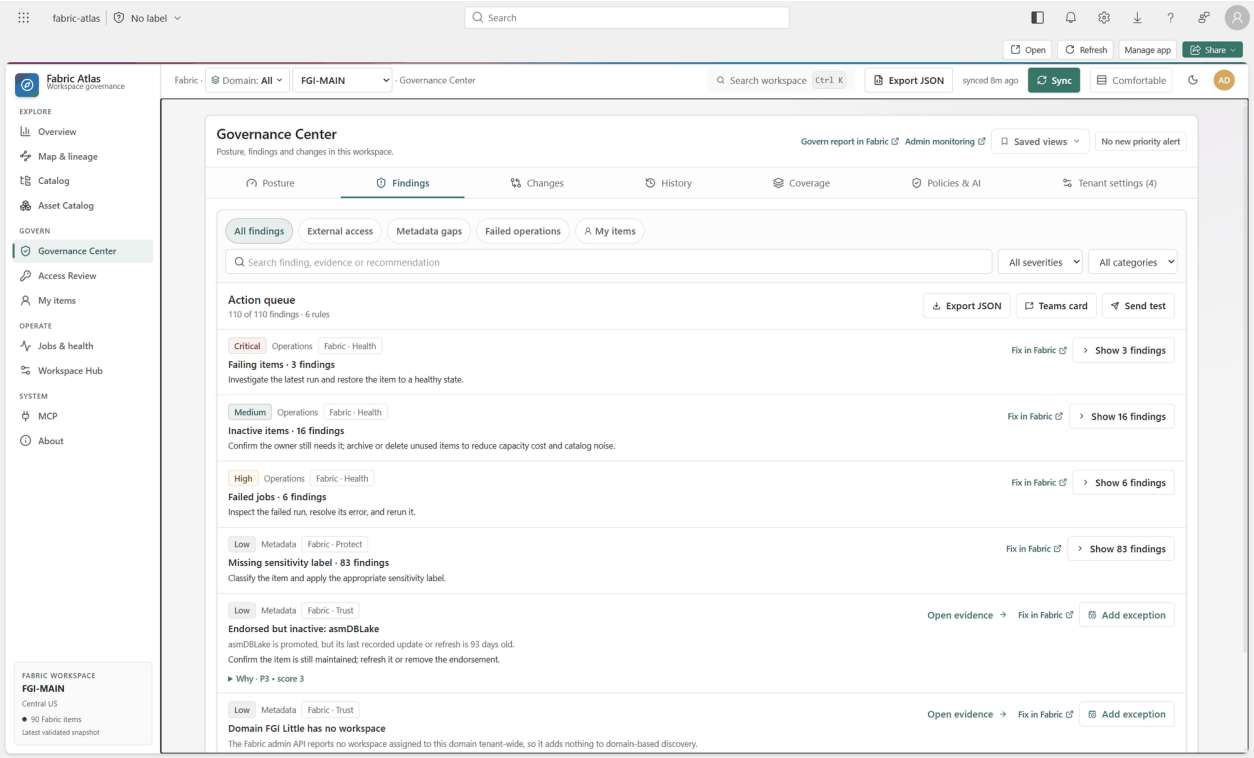


Figure 14: Current governance findings.

The captured action queue holds 110 findings from six rules, grouped by rule and ordered by priority: three failing items, six failed jobs, 16 inactive items and 83 items without a sensitivity label, plus individual findings such as an endorsed but inactive item and a domain without a workspace.

Findings remain visible until the underlying evidence changes. Personal acknowledgements and mutes do not hide the finding from other users and do not alter its raw score.

7.2 Change Center

Change Center compares two validated snapshots across item metadata, schema, access, sensitivity, lineage and jobs.

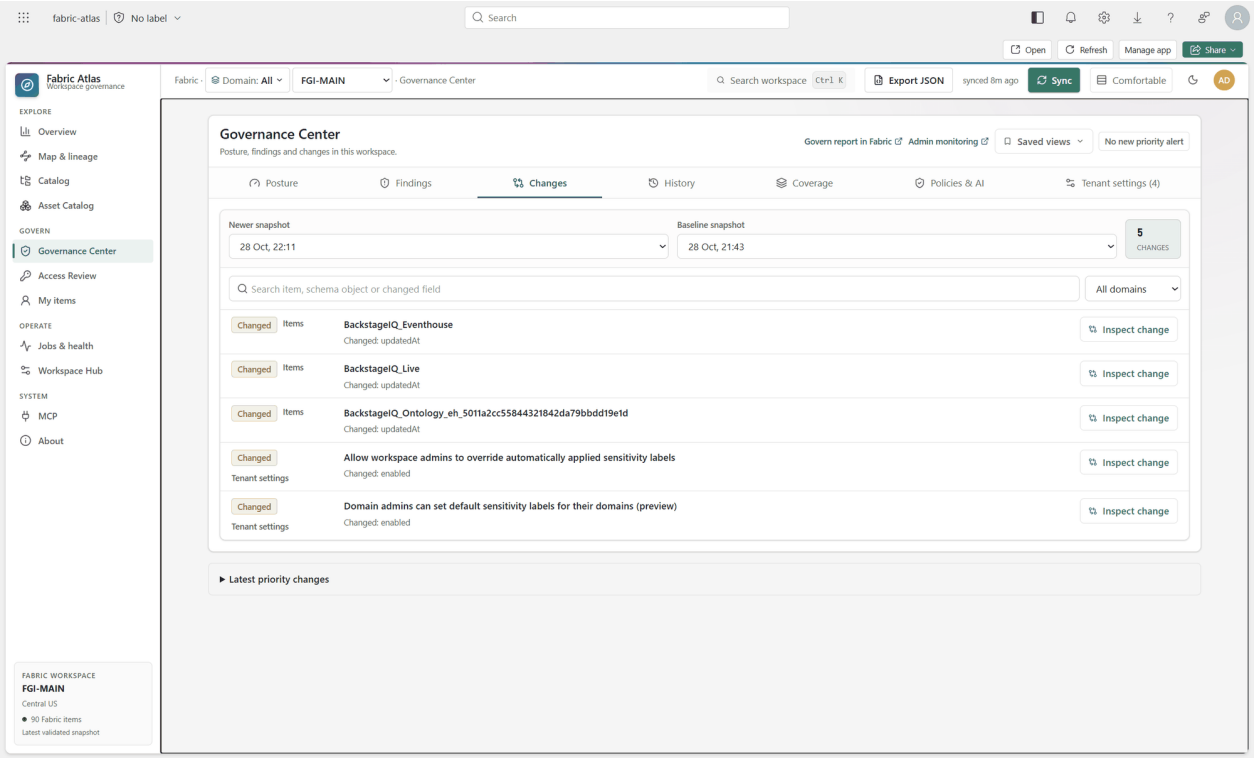


Figure 15: Snapshot comparison in Change Center.

The captured comparison reports five changes: three updated items and two tenant settings whose enabled state changed between the two snapshots.

Workspace manifests retain compact trend summaries. Older detailed catalogs are loaded only when the reviewer selects a comparison, which avoids hydrating every historical row at startup.

7.3 Metadata coverage

Coverage reports what Atlas collected, excluded, deferred or could not obtain through a verified public contract.

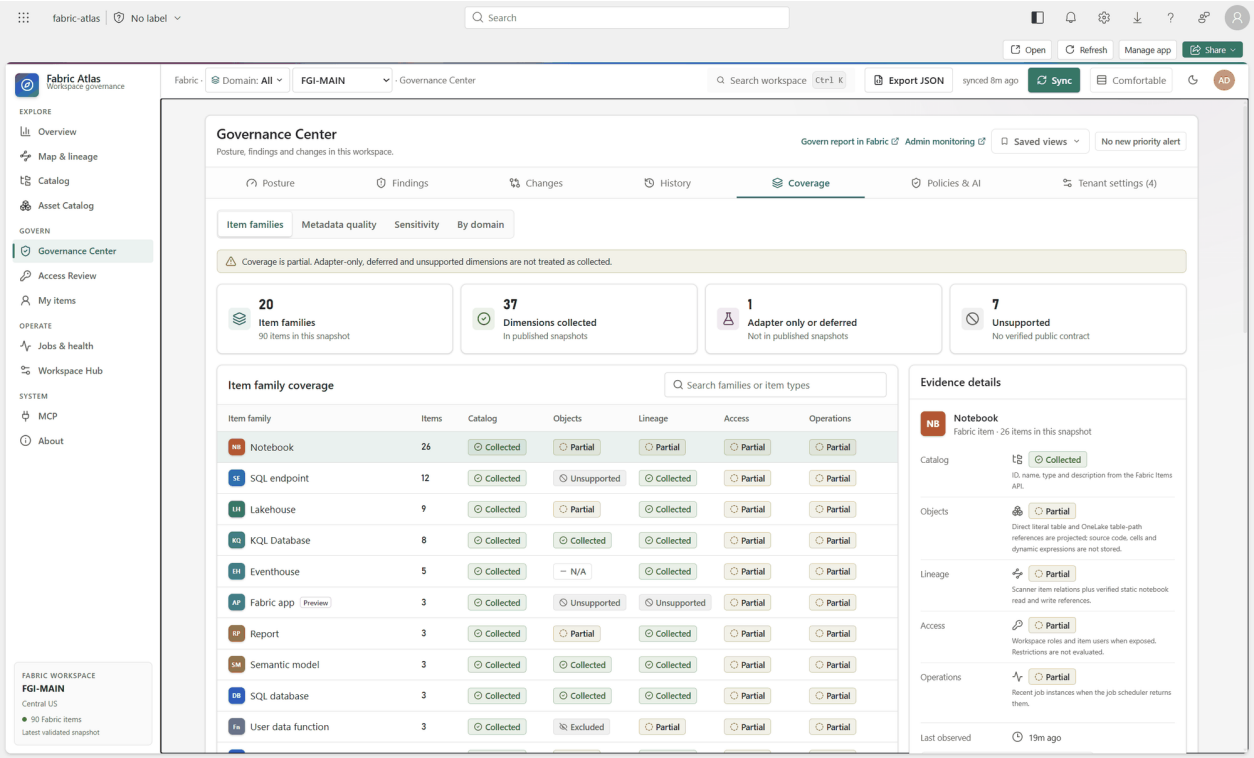


Figure 16: Item-family coverage and evidence detail.

The captured snapshot covers 20 item families and 37 collected dimensions. One dimension is adapter-only or deferred, and seven are unsupported. Coverage keeps catalog, object, lineage, access and operations depth separate for each item family.

This prevents two different states from being confused:

- Fabric returned the metadata family and the item has no value.
- Fabric did not expose the metadata family through the collection path.

7.4 Posture targets

Atlas scores six pillars independently: documentation, ownership, sensitivity, access, lineage and operations.

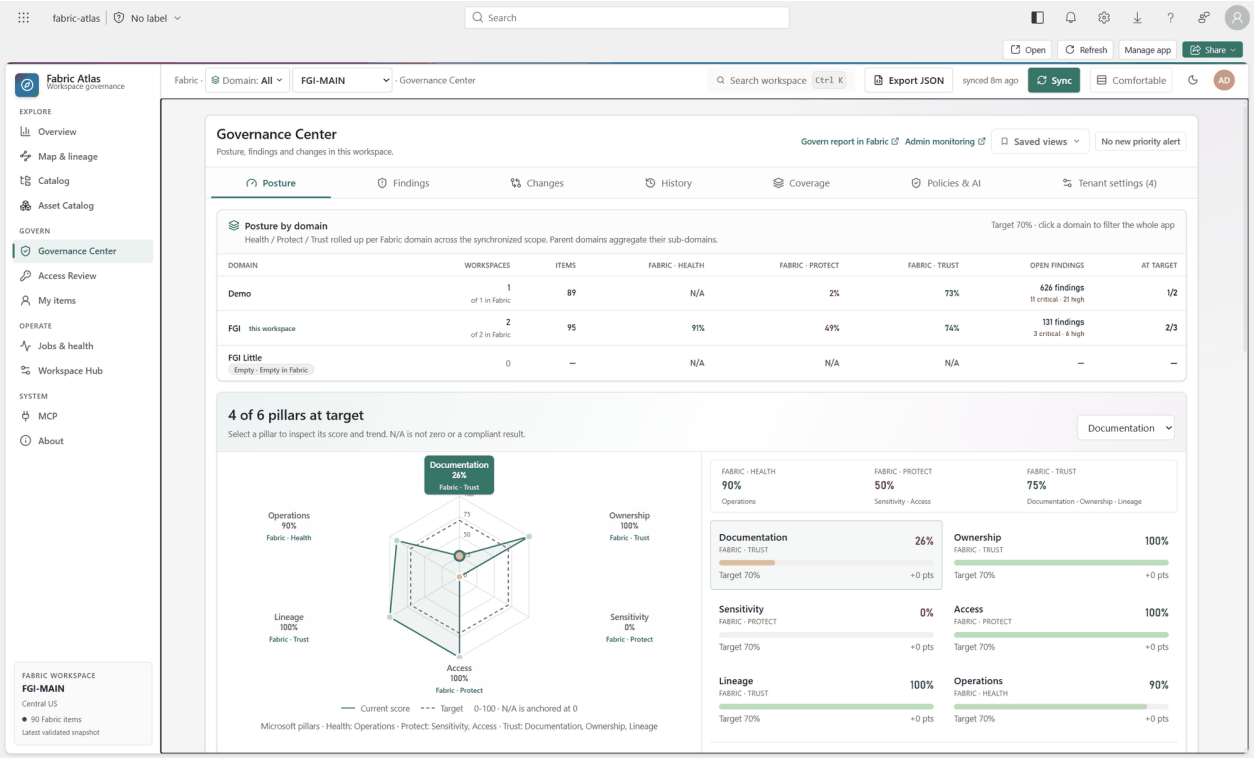


Figure 17: Four of six posture pillars at target.

The current scores are:

Pillar	Score	Target
Documentation	26%	70%
Ownership	100%	70%
Sensitivity	0%	70%
Access	100%	70%
Lineage	100%	70%
Operations	90%	70%

Non-applicable evidence is excluded rather than scored as zero. Every pillar links back to the items and findings behind the score.

The Posture tab opens with the same pillars rolled up per Fabric domain and mapped to the Microsoft Health, Protect and Trust groups. In this capture the FGI domain scores 91 percent Health, 49 percent Protect and 74 percent Trust, with two of three pillar groups at target. Fabric governance signals beside the Governance Radar report 2 percent endorsed items and 128 of 182 governance-relevant tenant settings enabled.

7.5 Policies & AI

Policies & AI lists semantic models, Data Agents, source metadata and protection evidence without inventing an exposure verdict.

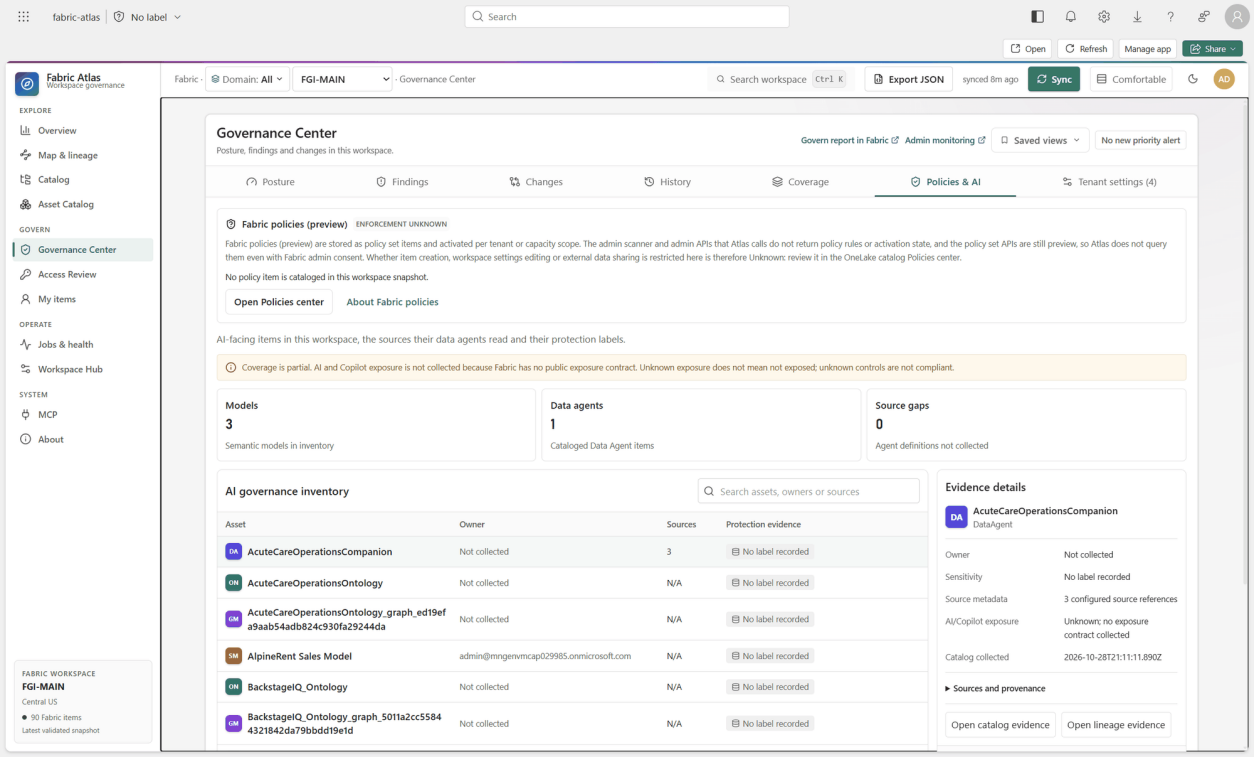


Figure 18: Policies and AI evidence for FGI-MAIN.

The captured inventory contains three semantic models and one Data Agent. The Data Agent has three configured source references and no missing definition. No sensitivity label is recorded for the displayed assets.

Fabric Policies evaluation is stored separately from recorded grants. If the collector is off or the tenant contract is unavailable, Atlas labels that evidence unavailable. Unknown exposure does not mean not exposed, and an unknown control is not presented as compliant.

8 Governance extensions in 2.6

Version 2.6 closes the quick-win gaps identified against the unified governance experience of Fabric and the OneLake catalog. All additional evidence comes from one bounded collector, `workspace-CollectGovernance`. Each section is optional and degrades to Unknown when an API, permission or tenant contract is unavailable.

8.1 Domains and catalog discovery

Capability	Behaviour
Domain names and posture	Workspaces show their Fabric domain name. Governance aggregates posture and metadata coverage per domain and can filter every view by domain
Catalog facets	The Catalog filters by type, owner, endorsement, sensitivity label, domain and tag, with a tag scope selector
Search snippets	Workspace search shows a short description excerpt beside each matching item
Request access	Items the signed-in user cannot open link to the Fabric request-access flow instead of a dead end
Identity sheet	The Catalog item drawer exports a Markdown one-pager with identity, ownership, trust, lineage, access, findings and recent jobs
My items	A personal scope filters catalog and findings to items the signed-in user owns
Snapshot export	Governance › Export snapshot downloads the active validated snapshot with a fingerprinted manifest for offline review

8.2 Explainable findings

Every finding carries a deterministic priority score: severity weight × impact factor × endorsement factor. The impact factor grows with downstream items, capped at ten, and certified or promoted items weigh more. The Why panel shows every input, the evidence identifiers, the downstream items and the recommendation. Priority orders work; it does not claim business risk.

New rules extend the catalogue:

Rule	Severity	Trigger
`workspace-no-admin`	Medium	The complete workspace role collection contains no Admin
`domain-default-label-missing`	Medium	An item in a domain with a default sensitivity label has no label
`item-inactive`	Low	A non-endorsed item has no recorded activity for `VITE_ATLAS_INACTIVE_DAYS` days, 90 by default

Rule	Severity	Trigger
`workspace-public-network`	Low	Inbound public access is allowed while the workspace holds Highly Confidential items; never raised when network state is Unknown
`external-share-sensitive`	Medium or High	A labelled item is shared with an external principal; High when the label is Highly Confidential

8.3 Tenant drift and Fabric policies

Atlas stores the tenant settings visible to the collector with each snapshot, and Change Center reports settings that were added, removed or changed between snapshots. A Fabric policies card lists the Fabric governance policies that the preview APIs expose. Enforcement is shown as Unknown because no public contract confirms evaluation results. Open in Fabric cards link to the matching Govern and Admin portal pages rather than reproducing their controls.

8.4 Notifications and AI assistance

Incremental synchronization re-enriches only items whose Fabric modification date changed and reuses the others; a missing date, a missing previous snapshot or a schema generation change forces a full run. The run records its mode and counters. When the `ATLAS_TEAMS_WEBHOOK_URL` secret is set, a Rayfin Function posts a card to Teams for Critical findings absent from the previous snapshot. When `ATLAS_AI_*` secrets are set, Atlas can suggest descriptions for undocumented items and tables. Only names, schema and DAX expressions are sent, capped at 24,000 characters and 50 targets per request, and every suggestion requires human review before use.

9 What's new in 2.7

9.1 Atlas Scout

Atlas Scout lets anyone ask governance questions in plain language. It is a chat client for a published Fabric Data Agent that reads the Atlas SQL database, so every answer comes from the same validated snapshots as the rest of the app. The full page opens from Govern › Atlas Scout, and a floating Scout bubble is available on every page. The Catalog item drawer has an **Ask Scout** button that opens the bubble with a question about that item.

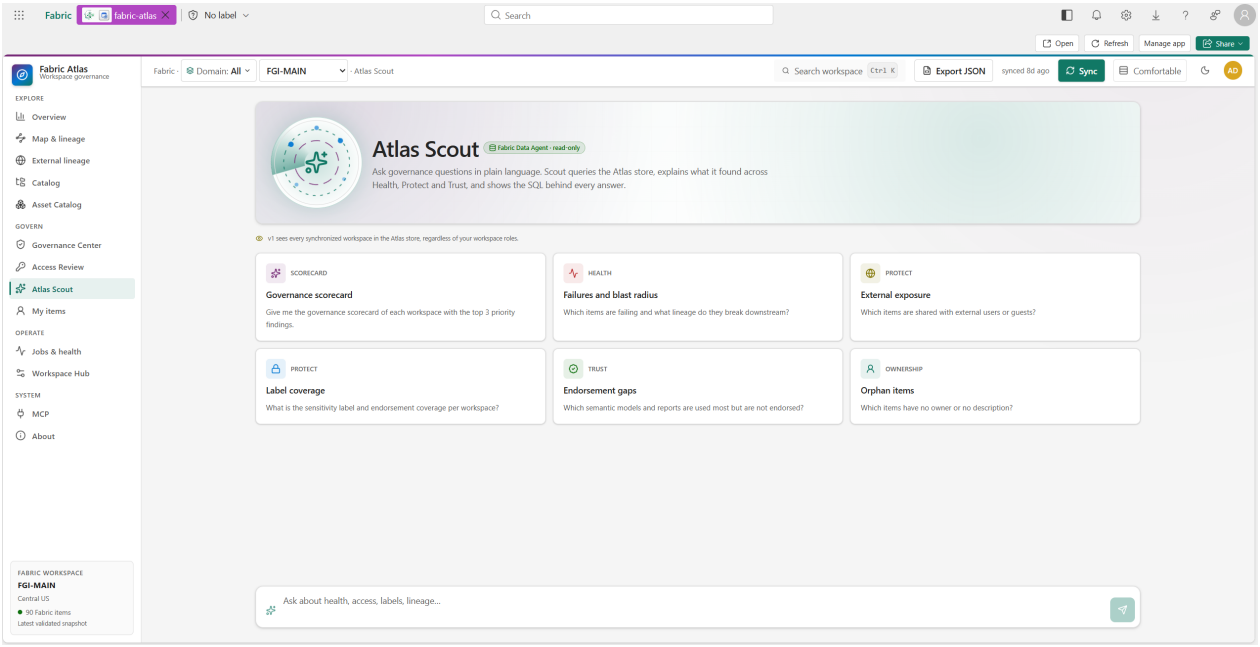


Figure 19: Atlas Scout home with starter questions grouped by Health, Protect, Trust and Ownership.

The agent follows the OneLake catalog pillars (Health, Protect, Trust) plus Ownership. It can build a workspace scorecard, find failures and their blast radius, list external exposure, measure label and endorsement coverage and find orphan items. Besides items, runs, findings and access, it reads the object inventory captured by the sync: tables, columns, measures with their DAX dependencies, relationships, ontology bindings and object-level lineage, plus domains, OneLake roles and tenant settings.

Answers are rendered as headings, numbered findings, priority pills, health gauges and sortable tables. Every answer can show the SQL the agent ran and the raw result rows. When an answer contains source and target columns, Scout draws a lineage graph and highlights broken links and failing items.

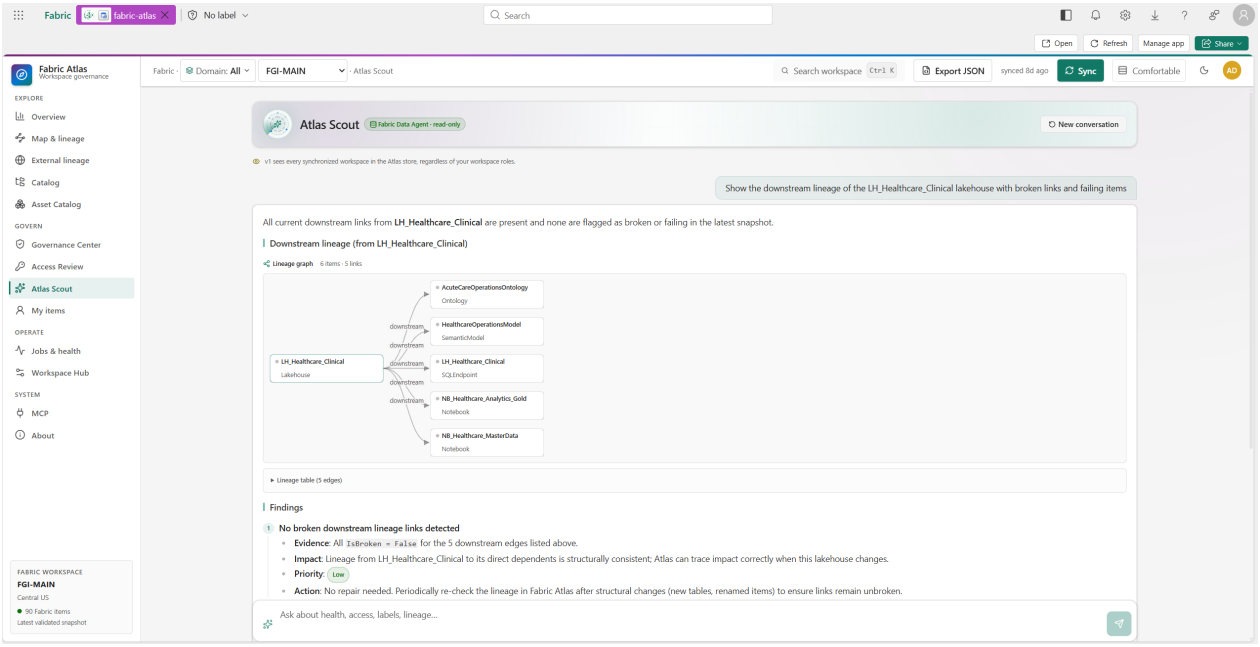


Figure 20: Atlas Scout drawing the downstream lineage of a lakehouse, followed by a prioritized finding.

Scout shows each step of the run: secure session, understanding, querying the Atlas store and writing the answer. Stop cancels the agent run. If the service still reports a run as active, Scout cancels it and continues in a fresh thread. Scout waits up to five minutes for an answer: an item question runs four or five SQL steps in sequence, about 30 seconds each.

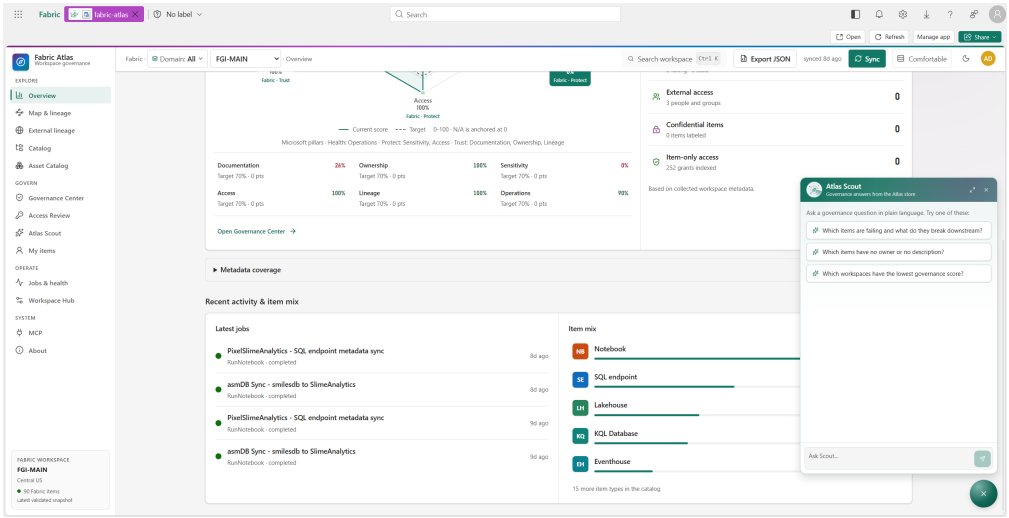


Figure 21: Floating Scout window with suggested questions.

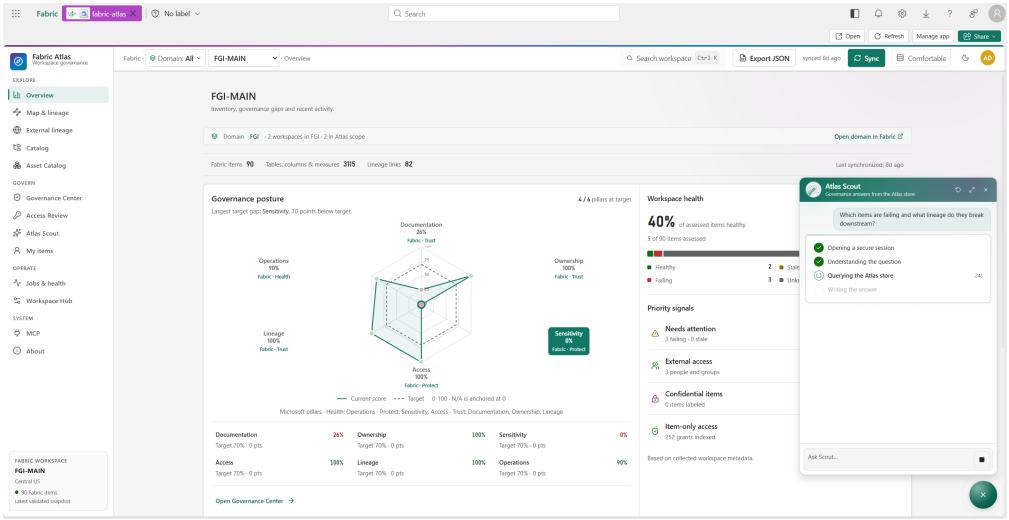


Figure 22: Floating Scout window showing live progress while the agent queries the store.

The agent definition, instructions, example queries and provisioning script are versioned in `fabric/data-agent/atlas-scout`. Scout v1 is read-only and sees every synchronized workspace in the Atlas store, regardless of the user's workspace roles. First use asks for consent to `DataAgent.Execute.All` unless an administrator granted it tenant-wide.

9.2 External lineage (OpenLineage)

Atlas imports OpenLineage files (NDJSON or JSON) produced by sources outside Fabric, such as databases, ETL tools and streams. Datasets whose namespace and name exactly match a Fabric item are linked automatically; others can be linked manually to any catalog item. Imports are stored outside the immutable snapshot as `OpenLineageImport`, `ExternalDataset` and `ExternalLineageEdge` rows and never keep the raw file.

On Map & lineage, an External lane shows the imported jobs and datasets with link points on Fabric items. Impact follows the lineage across those link points, and the impact report lists external upstream and downstream datasets and the jobs between them. Atlas can also export a workspace's lineage as static OpenLineage `JobEvent` and `DatasetEvent` records carrying the custom `fabricAtlas_item` facet.

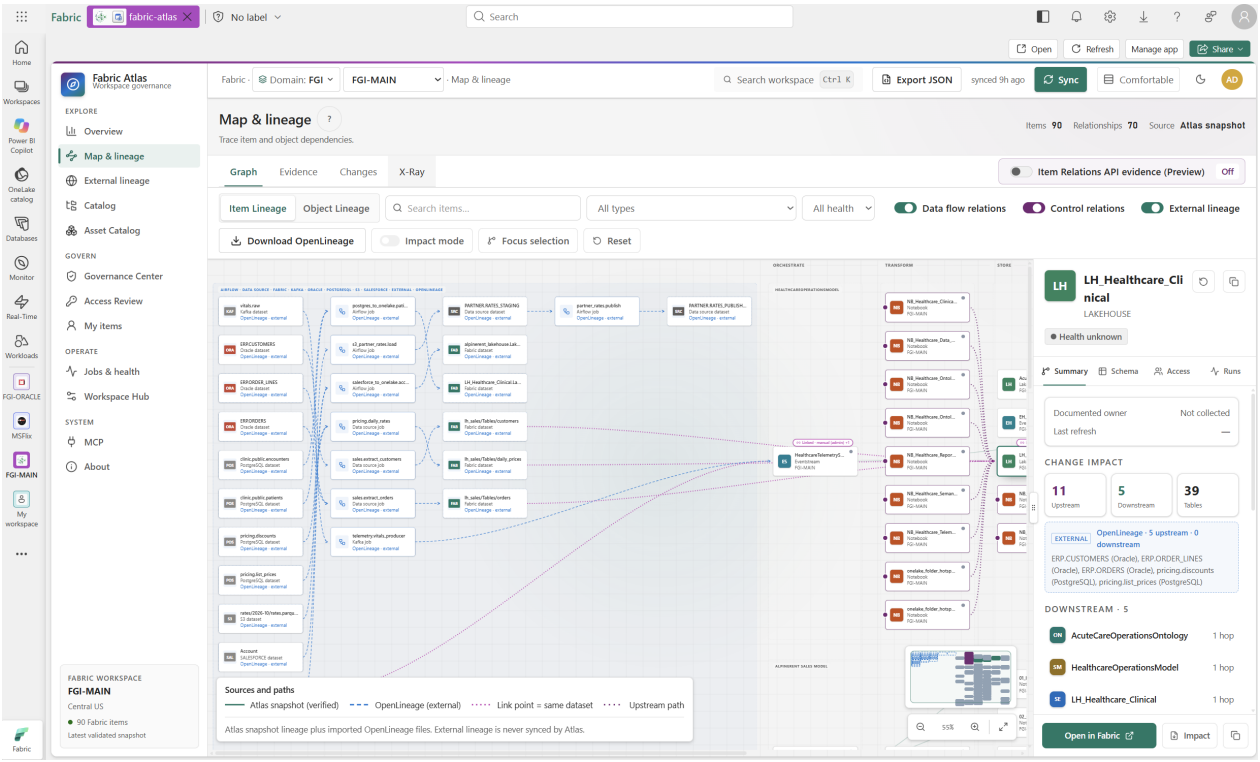


Figure 23: Map & lineage with the External lineage lane.

9.3 Sync Brief

The Sync Brief compares the latest validated snapshot with the previous one. It lists items added or removed, new external or elevated access, lineage edges added or removed, jobs that failed or recovered, changed measures and schema objects, new high-priority findings and the largest downstream impact. A section without evidence is marked unavailable rather than empty. The brief exports as Markdown for sharing outside Atlas.

9.4 Notebook object lineage

Notebook Object Lineage shows direct literal table references found in the notebook definition, with read and write direction. The collector parses the definition in memory and publishes only bounded object names, operations and source item IDs. It never stores notebook cells, source code, SQL text, credentials or dynamically constructed references. When no direct object name can be resolved, the view falls back to the notebook's verified item-level dependencies.

10 Access review and departure planning

10.1 Review matrix

Fabric permissions are additive. A workspace role can grant access to every item while a direct item share adds another path. Atlas calculates the strongest recorded permission for every reachable principal-item pair.

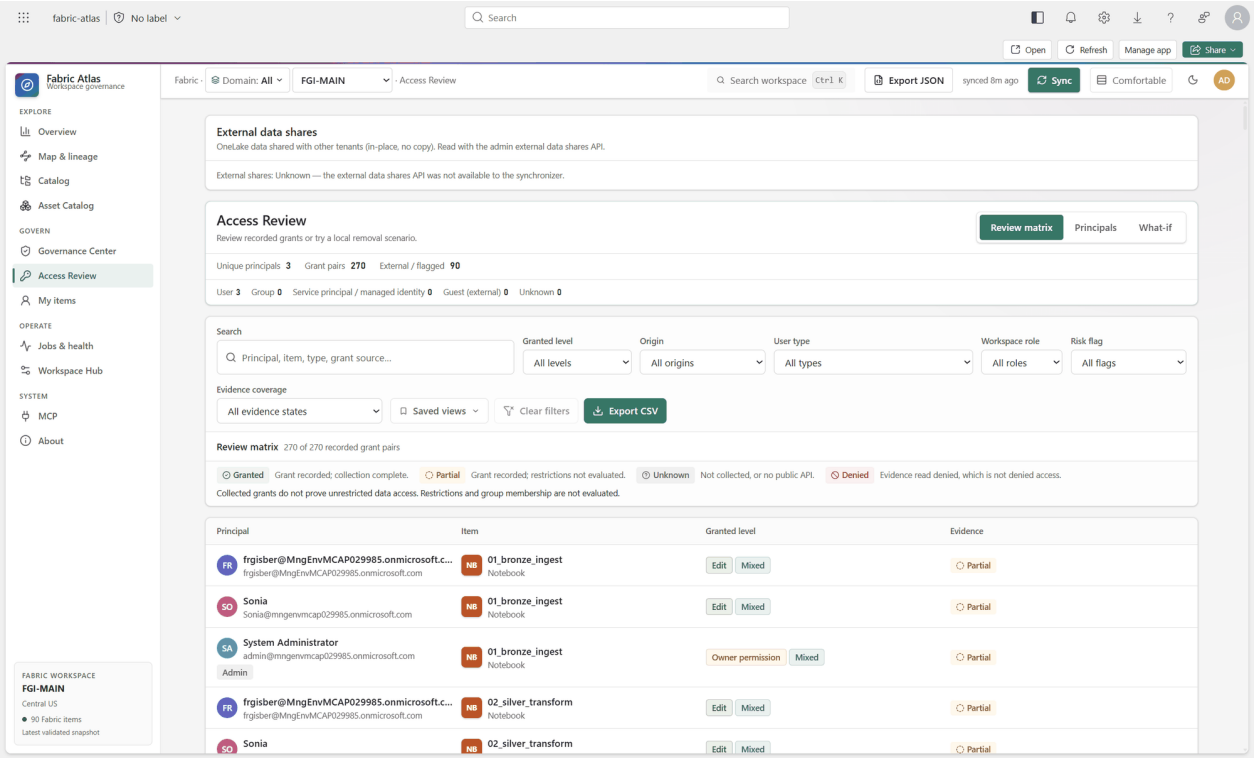


Figure 24: Access Review matrix with evidence coverage.

The captured workspace contains three resolved principals and 270 recorded principal-item pairs. Filters cover level, origin, risk flag and evidence coverage. The matrix can be exported as CSV.

Recorded grants do not prove unrestricted data access. Group membership, OneLake security, Purview DLP and Fabric Policies are shown as collected, unavailable, unsupported or not evaluated. Since 2.6, filters also cover principal type and role, and OneLake data access roles are collected for up to 40 Lakehouses with direct members and the presence of row-level or column-level security. Constraint text is never stored and groups are not expanded.

10.2 Principal-centred review

Principals mode groups the filtered pairs by identity and strongest recorded permission.

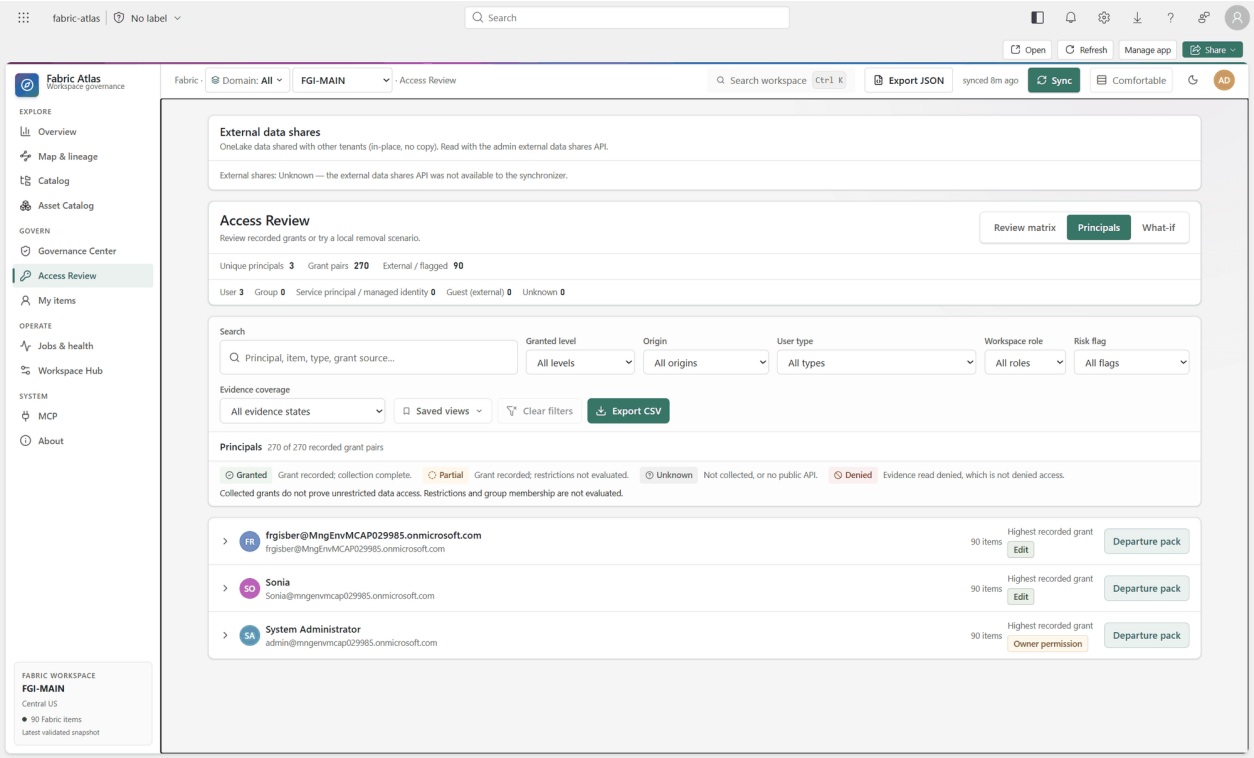


Figure 25: Access Review grouped by principal.

Personal review decisions are stored per authenticated user. Reviewed, Accepted and Needs action events do not become a team-wide approval.

10.3 Read-only What-if

What-if removes selected recorded grant paths in memory. It does not call a Fabric write API.

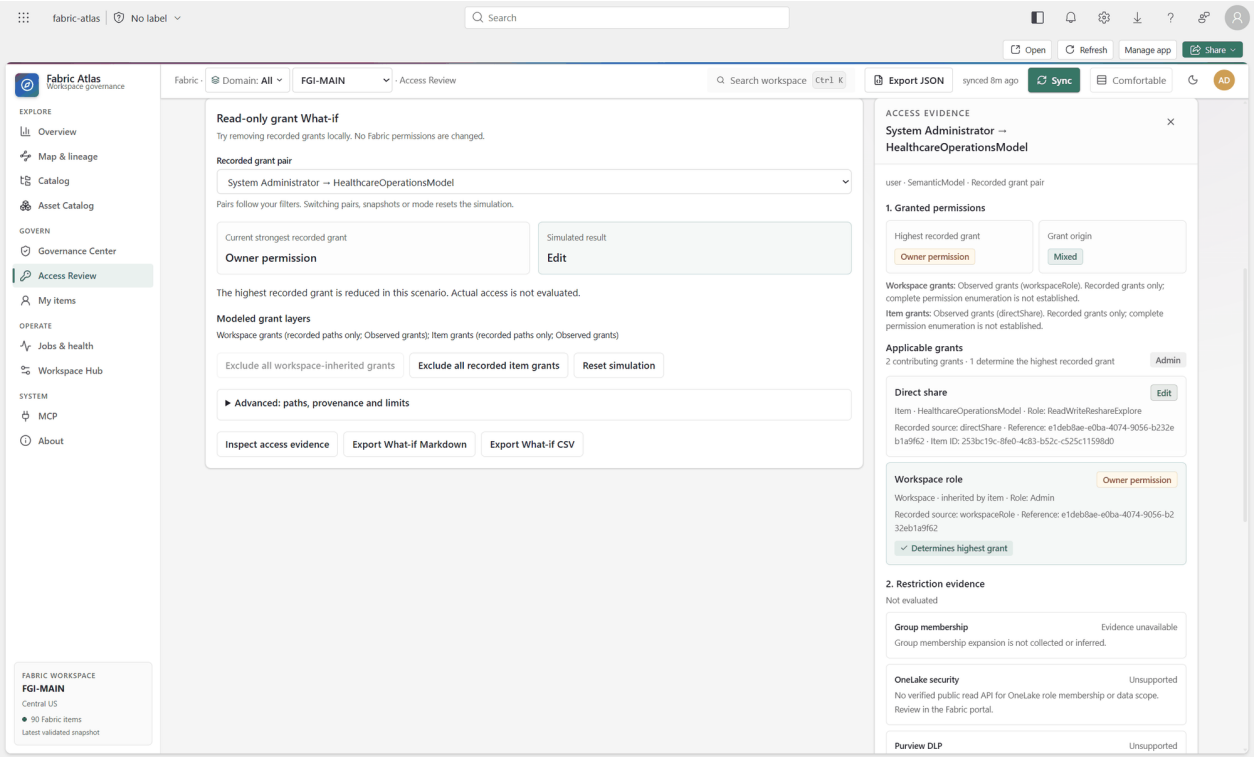


Figure 26: A local What-if scenario for HealthcareOperationsModel.

The selected pair has Owner permission through the workspace Admin role and an Edit-level direct item share. Excluding workspace-inherited grants lowers the simulated result to Edit. The panel states that actual access is not evaluated because restriction and group-membership layers remain outside this grant-only model.

The scenario can be exported as Markdown or CSV with the evidence boundary attached.

10.4 Departure packs

A Departure pack combines ownership, effective owner access, sole ownership, downstream blast radius and reassignment evidence for a person or guest.

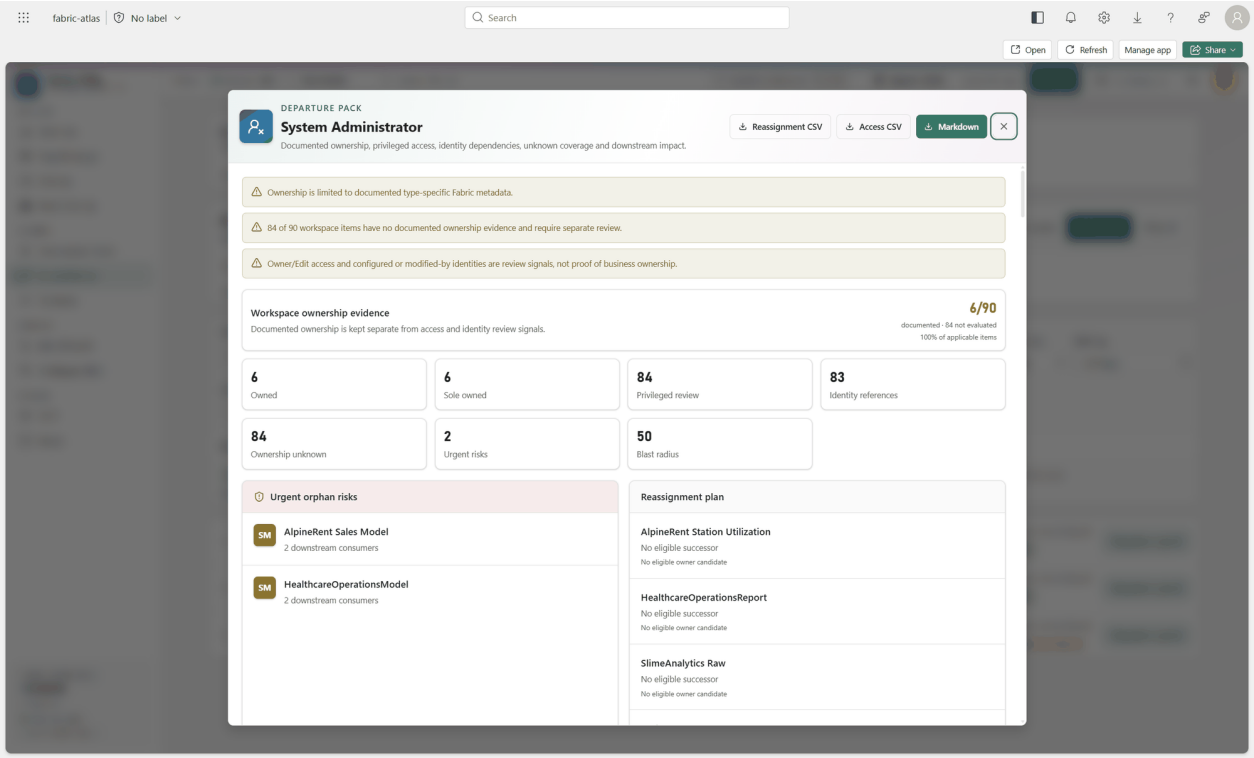


Figure 27: Departure pack for the System Administrator.

In this capture, six of 90 items have documented ownership evidence for the System Administrator, all six solely owned. Two semantic models are flagged as urgent orphan risks because each has two downstream consumers, and 84 items with direct Owner access are listed for privileged review.

Ownership conclusions require type-specific Fabric metadata. A matching display name alone is not accepted as proof. If no eligible successor can be supported by the graph and access evidence, the report states that no candidate was found.

Exports include reassignment CSV, effective-access CSV and a complete Markdown pack.

11 Operations and workspace context

11.1 Jobs & health

Jobs & health separates observed execution evidence from downstream items inferred through stored lineage.

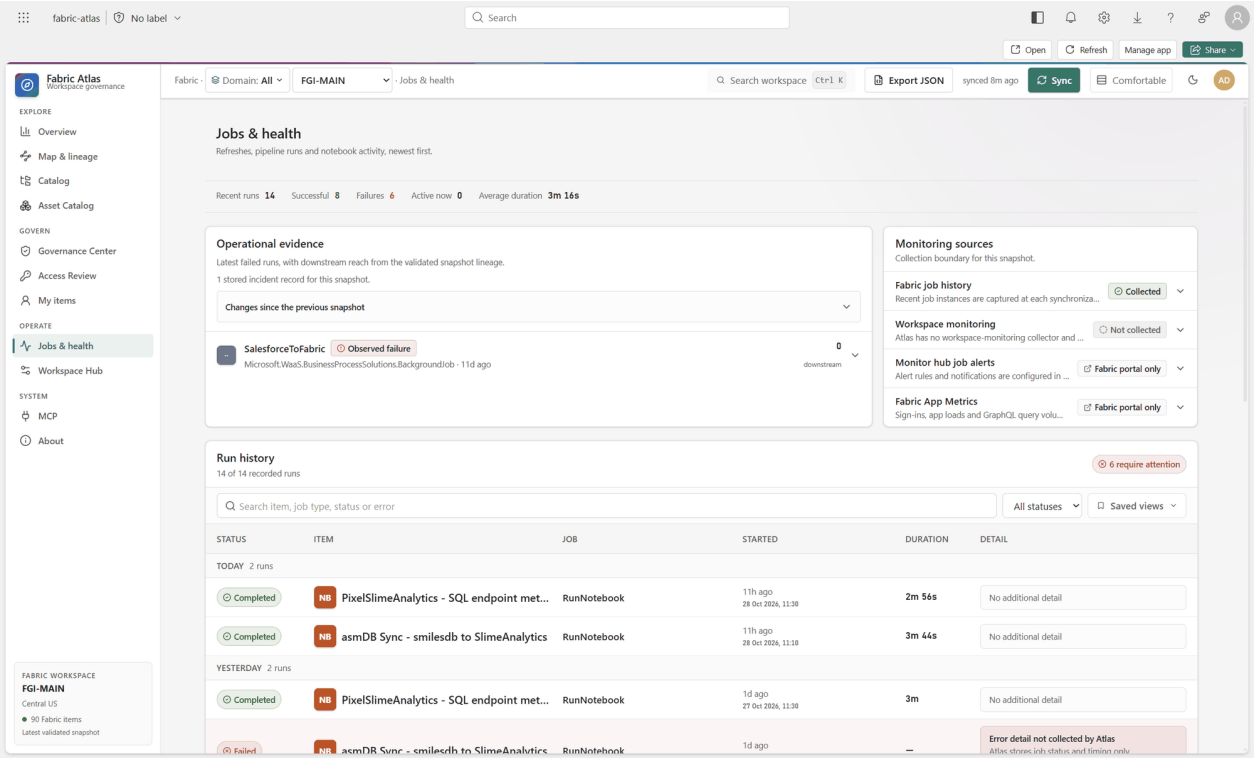


Figure 28: Jobs, operational evidence and monitoring boundaries.

The captured snapshot contains 14 recent runs: eight successful and six failed, with an average duration of 3 minutes and 16 seconds.

Monitoring sources are explicit:

- Fabric job history is collected.
- Workspace monitoring is not collected by Atlas.
- Monitor Hub alerts remain portal-only.
- Fabric App Metrics remain portal-only.

An optional monitoring gap does not invalidate a trusted catalog snapshot.

11.2 Multi-workspace scope

The synchronizer chooses which workspaces belong to the shared Atlas scope. Unselected discovery results are never stored.

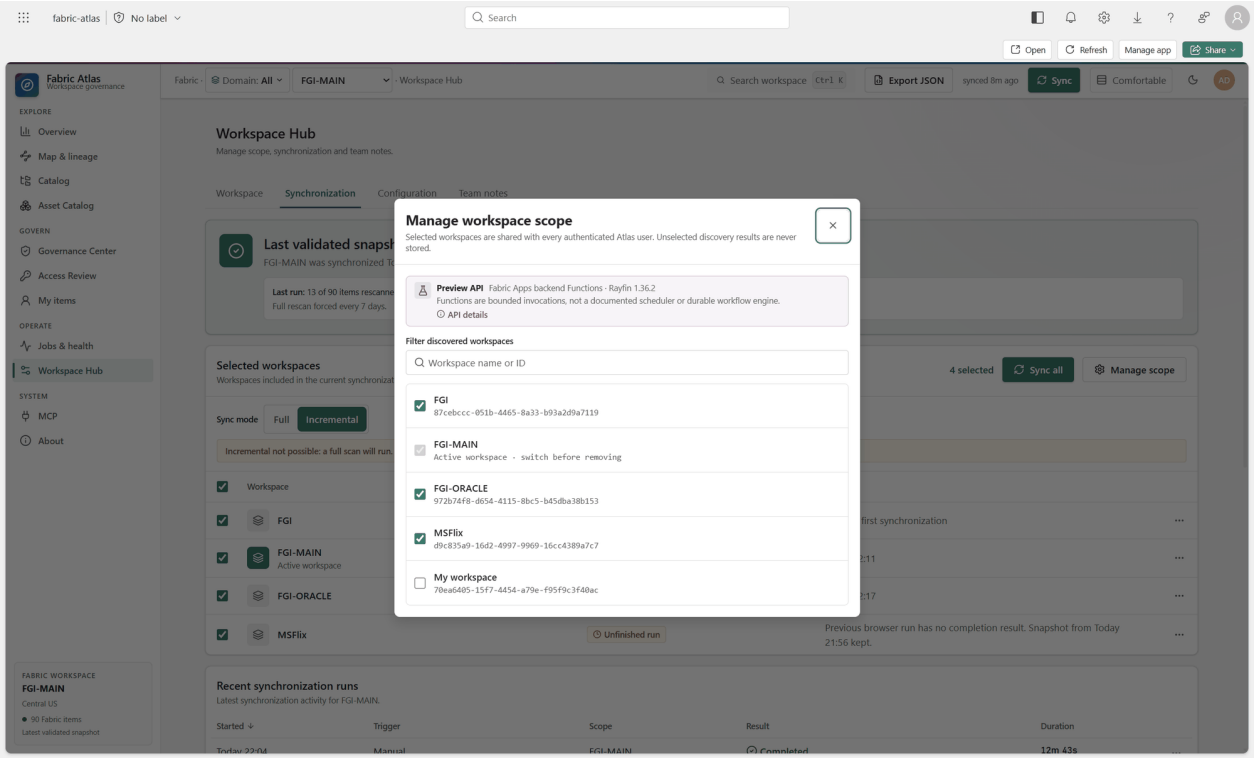


Figure 29: Multi-workspace discovery and shared scope selection.

FGI, FGI-MAIN, FGI-ORACLE and MSFlix are selected. The dialog also shows unselected workspaces returned by discovery. The active workspace cannot be removed until the user switches away from it.

Every authenticated app user can read the selected catalog scope. Only the configured synchronizer can change the shared scope or publish snapshots.

11.3 Synchronization

Each workspace has an independent synchronization state and manifest.

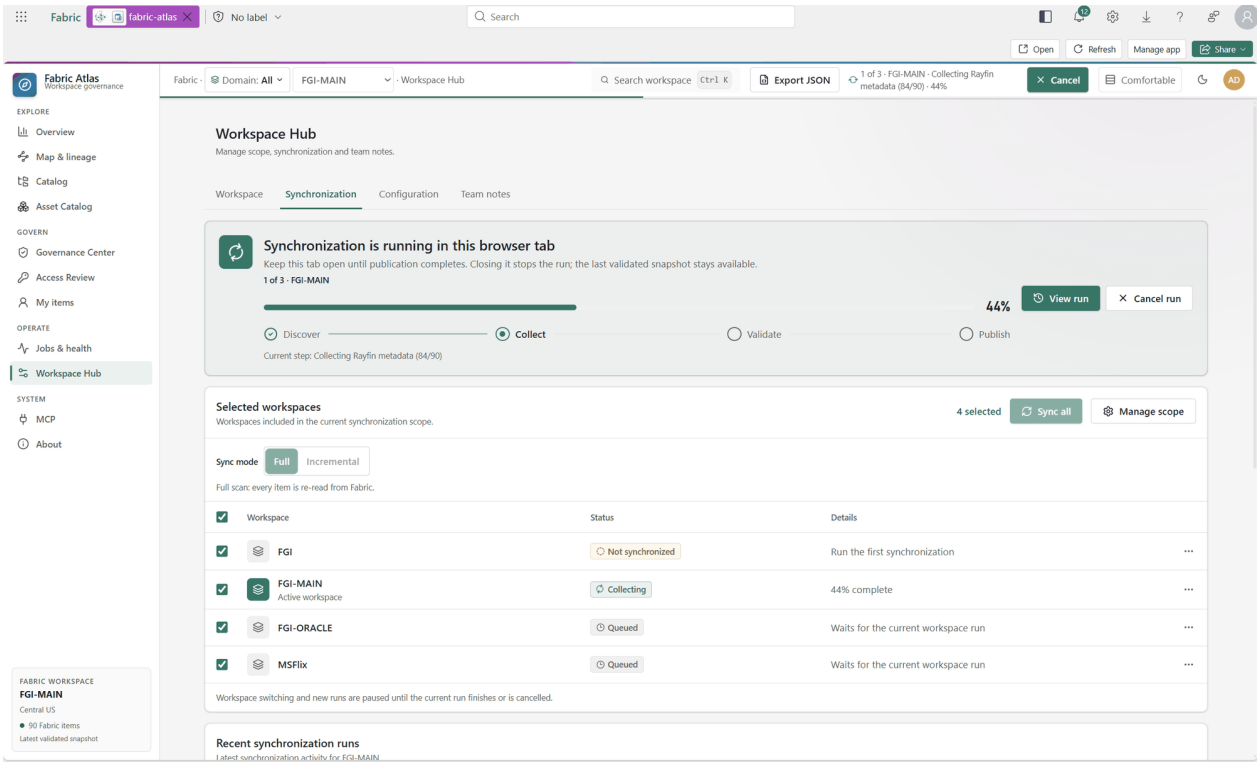


Figure 30: Synchronization state for the selected workspace scope.

The panel shows selected workspaces, snapshot status, manual run history and schedule state. In this capture a multi-workspace run is in progress: FGI-MAIN is the first of three workspaces and is 44 percent complete. Each workspace keeps its last valid snapshot until its own run publishes a new manifest.

Runs can be full or incremental. The last FGI-MAIN incremental run rescanned 13 of 90 items and reused 77 in 12 minutes 43 seconds, against 8 minutes 59 seconds for the preceding full run. A full rescan is forced every seven days.

Scheduled refresh is disabled. Fabric Apps backend Functions do not expose a documented timer or unattended trigger, and the collectors need a delegated identity that Atlas never stores.

11.4 Configuration

Workspace Hub groups synchronized configuration by item and section instead of showing one unstructured JSON payload.

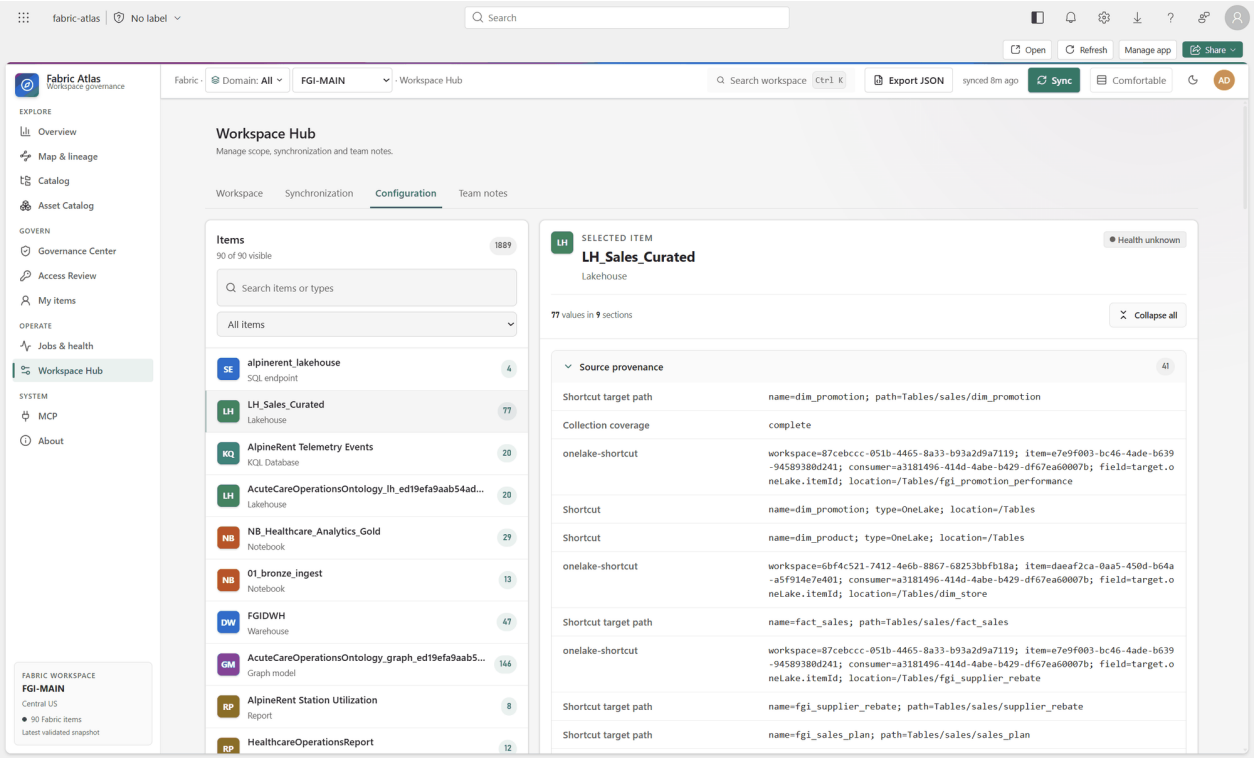


Figure 31: Synchronized configuration in Workspace Hub.

Configuration includes bounded schema chunks, endpoint context, definitions and source provenance. Target URLs, source rows, credentials and executable source code remain outside the snapshot.

11.5 Team notes

Team notes attach shared operational context to the workspace or a selected Fabric item. The capture shows the workspace thread before any note is posted.

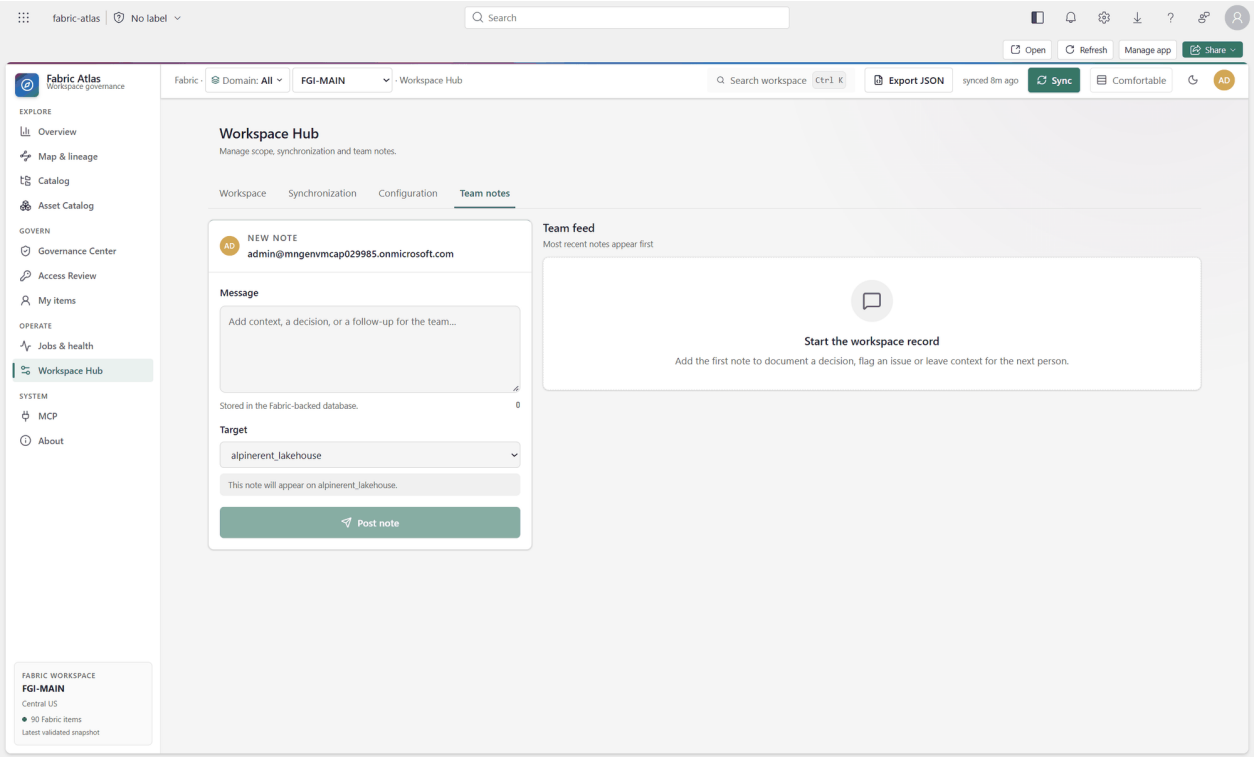


Figure 32: Shared team notes in Workspace Hub.

Notes are append-only. Update and delete actions remain unavailable until Atlas has a reviewed policy and user experience for shared-note editing.

12 Architecture and synchronization

Fabric Atlas is a React and Vite application deployed through Rayfin inside the Fabric portal. Fabric brokered authentication supplies the signed-in Entra identity.

```mermaid
flowchart LR
  U["Fabric user"] --> P["Fabric portal"]
  P --> A["Fabric Atlas\nReact application"]
  A <--> AUTH["Fabric brokered\nauthentication"]
  A --> C["Browser sync coordinator\nworkspace and item batches"]
  C --> RF["Typed Rayfin Functions\nbounded collectors"]
  C --> PY["Python compatibility UDF\ndocumented gaps only"]
  RF <--> FAPI["Fabric REST and\ndefinition APIs"]
  RF <--> DATA["Read-only SQL and\nKusto metadata endpoints"]
  PY <--> PBI["Power BI admin scanner\nand PBIR-Legacy pages"]
  PY <--> DATA
  RF --> V["Contract validation\nfailure isolation and merge"]
  PY --> V
  V --> R["Rayfin Data API"]
  R --> DB[("Fabric SQL database\nworkspace snapshots and notes")]
  V --> M["Workspace manifest\npublished last"]
```

```
M --> DB

classDef user fill:#742774,stroke:#742774,color:#ffffff;
classDef portal fill:#0078d4,stroke:#005a9e,color:#ffffff;
classDef app fill:#117865,stroke:#0c695a,color:#ffffff;
classDef auth fill:#f7eff8,stroke:#742774,color:#242424;
classDef function fill:#eef6fc,stroke:#0078d4,color:#242424;
classDef api fill:#e3f7ef,stroke:#117865,color:#242424;
classDef data fill:#fffef5,stroke:#817400,color:#242424;

class U user;
class P portal;
class A,C,V app;
class AUTH auth;
class RF,PY function;
class FAPI,PBI,DATA api;
class R,DB,M data;
```

12.1 Collector ownership

Evidence	Primary collection path	Compatibility path
Workspaces, items, roles and jobs	`workspaceCollectCore`	Explicit Python rollback only
Definitions and structural metadata	Rayfin definition, Power BI, KQL and SQL collectors	Semantic-model scanner fallback when a definition is unavailable
Lakehouse, Warehouse, SQL Database and Mirrored Database objects	`workspaceCollectSqlMetadata` plus Lakehouse REST	None; missing application-identity coverage stays explicit
Shortcuts, mirroring and materialized lake view provenance	`workspaceCollectSourceProvenance`	None
Item Relations evidence	`workspaceCollectItemRelations`	None; the API remains Beta and non-authoritative
Access-policy context	`workspaceCollectAccessPolicyEvidence`	Policy layers remain unavailable
Power BI admin scanner and scanner lineage	Python UDF	Retained because Rayfin Functions have no documented Power BI audience
PBIR-Legacy report pages	Python UDF	Retained until a supported application-identity path exists
Kusto live schema	Python UDF	Retained because Rayfin Functions have no documented Kusto audience

12.2 Why one synchronization can take several minutes

The browser sends bounded collector calls instead of one unbounded function request. Each call has deadlines, pagination, retries, cancellation and response-size limits. Item failures are isolated so one optional collector cannot erase valid metadata from another item.

The browser publishes nothing until every required collector has completed and the merged snapshot passes validation.

12.3 Immutable snapshots

Child rows are written first. The Workspace manifest is written last. Only that manifest makes the new snapshot visible.

```
flowchart LR
  S["Create running\nSyncRun"] --> D["Discover selected\nworkspace scope"]
  D --> B["Run bounded Rayfin\ncollector batches"]
  B --> P["Run exact Python\ncompatibility plan"]
  P --> V{"Required contracts complete\nand item results valid?"}
  V -->|No| F["Mark attempt failed\npreserve active snapshot"]
  V -->|Yes| W["Write snapshot rows\nin bounded batches"]
  W --> R["Read rows back through\nproduction pagination"]
  R --> C{"Counts and identities\nmatch the manifest?"}
  C -->|No| F
  C -->|Yes| T["Mark SyncRun completed"]
  T --> M["Publish workspace\nmanifest last"]
  M --> A["Activate snapshot\nand apply retention"]

  classDef start fill:#742774,stroke:#742774,color:#ffffff;
  classDef process fill:#eef6fc,stroke:#0078d4,color:#242424;
  classDef decision fill:#fffef5,stroke:#817400,color:#242424;
  classDef safe fill:#e3f7ef,stroke:#117865,color:#242424;
  classDef fallback fill:#f7eff8,stroke:#742774,color:#242424;

  class S start;
  class D,B,P,W,R,T,M process;
  class V,C decision;
  class A safe;
  class F fallback;
```

If collection, cancellation, persistence or reconstruction fails, child rows without a complete manifest remain invisible. Hydration loads the newest trusted snapshot for that workspace.

12.4 Retention and writer trust

Retention defaults to 12 validated snapshots and can be configured from 2 to 50. Cleanup starts only after the new manifest is published.

Snapshot publication is bound to the configured immutable Entra subject. Email remains the visible contact and historical writer label, but it is not the authorization key. Former synchronizers can be retained for historical reads and controlled cleanup without being allowed to publish.

12.5 Persisted entities

Entity	Stored purpose
Workspace	Workspace-scoped manifest and compact governance summary
FabricItem	One row per Fabric item
LineageEdge	Directed source-to-consumer dependency
Principal	User, group, service principal or guest identity
AccessGrant	Workspace or item access evidence
JobRun	Recent refresh, pipeline and notebook activity
ConfigEntry	Bounded configuration facts, schema chunks and provenance
Comment	Shared workspace or item note
SyncRun	Running, completed or failed synchronization attempt
SavedView	Personal navigation and filter preset
AccessReview	Personal decision for one effective access pair
AccessReviewEvent	Personal append-only review history
FindingAck	Personal Governance Radar acknowledgement or mute
GovernancePolicy	Shared posture targets
GovernanceException	Shared, time-bounded exception with reason and scope
OpenLineageImport	One imported OpenLineage file: name, counts and result
ExternalDataset	External dataset, optionally linked to a Fabric item
ExternalLineageEdge	External input-to-output edge through a job

13 Security and data boundaries

13.1 Data Atlas stores

Atlas stores workspace and item identity, documented ownership, descriptions, labels, tags, object schema, measure expressions, lineage, principals, grants, job metadata, configuration facts, snapshot summaries and team notes.

13.2 Data Atlas excludes

Excluded content	Boundary
Table and event rows	Business data is outside the product purpose

Excluded content	Boundary
Data source credentials and connection payloads	Cataloging does not require secrets
Power Query and source expressions	Source logic is not copied
Notebook source and cells	The item is indexed without copying code
Pipeline activities and expressions	The item and exposed lineage are stored, not orchestration source
User Data Function source	Functions remain visible as items without copying implementation code
KQL rows, query text and policy bodies	Atlas reads structural metadata only
Mirroring source rows, source database names and target URLs	Only bounded provider, selection and replication context is retained
Complete report visual field bindings	The current collection path does not expose them reliably

Collector output passes through explicit contracts and allowlists before persistence. Unexpected fields are not serialized by default.

13.3 Authentication and authorization

Fabric brokered authentication runs the app under the signed-in Entra identity. Collection tokens must match the current Fabric user and tenant.

State	Read scope	Write scope
Selected workspace catalog and history	Every authenticated user admitted to the deployed app	Configured synchronizer only
Shared team notes	Every authenticated app user	Authenticated author with email and subject binding
Saved views	Current user only	Current user only
Access review decisions	Current user only	Current user only
Governance Radar acknowledgements and mutes	Current user only	Current user only

The Fabric App audience is the catalog read boundary. Selected workspace metadata and team notes are shared with that complete audience. Personal review state remains user-scoped.

13.4 Failure behaviour

Atlas favours explicit failure over success-shaped fallbacks:

- A required contract failure rejects the refresh.
- An incomplete snapshot never becomes active.
- Cancellation before manifest publication keeps the current snapshot.
- One workspace failure cannot invalidate another workspace manifest.

- Malformed workspace IDs, endpoint identities or authoritative lineage fail closed.
- Payloads above the configured bounds are rejected before persistence.
- Unknown sensitivity rankings do not create downgrade alerts.
- Ambiguous DAX references and ambiguous principals are omitted.
- Personalization failures do not hide the underlying governance evidence.

14 Deployment and operating model

14.1 Prerequisites

A live deployment requires:

1. A Microsoft Fabric workspace on supported capacity.
2. Fabric Apps enabled for the relevant tenant users.
3. A Rayfin deployment with Fabric authentication, data, storage, static hosting and Functions.
4. An Entra single-page application registration for delegated compatibility collection.
5. A Fabric Administrator for the Power BI scanner and its required tenant settings.
6. The minimal `atlas_sync_functions` UDF published for retained gaps.
7. One configured synchronization account.

14.2 Deployment flow

The release deployment command is:

```
npx rayfin login --tenant <tenant-id> --select
npx rayfin up --tenant <tenant-id> --workspace-id <workspace-id> --item-name fabric-atlas
```

Rayfin applies the MSSQL schema, deploys typed Functions, publishes the static app and writes runtime settings. The compatibility UDF must be updated by round-tripping its complete Fabric definition and replacing only `function_app.py`.

14.3 First sync and upgrades

The first deployment blocks catalog access until the configured synchronizer publishes a trusted snapshot. The same gate can reappear when a major or minor snapshot contract changes.

If another user reaches the gate, Atlas keeps the configured synchronizer contact visible.

14.4 Routine operation

1. Restrict the Fabric App audience to people allowed to read the selected workspace scope.
2. Keep the synchronizer identity and compatibility permissions current.
3. Run synchronization after material workspace changes.
4. Review failed items and optional collector boundaries.
5. Review the Governance Radar and Change Center after a new validated snapshot.
6. Use Coverage before treating a missing value as a governance defect.
7. Rotate the trusted writer explicitly when responsibility changes.

Scheduled and durable unattended refresh remain blocked by platform contracts. Atlas does not store a delegated user token to work around that limit.

15 Fabric item coverage

Atlas always indexes top-level items. Deeper inventory depends on item type, tenant settings and the endpoints Fabric exposes.

Fabric element	Current depth	Important boundary
Lakehouse	Item metadata, tables, columns, SQL endpoint context, shortcuts, materialized lake views and lineage	Schema-enabled variants use SQL endpoint metadata when the legacy tables route returns HTTP 400
Warehouse	Item metadata, SQL tables, views, columns, access, lineage and jobs	SQL endpoint visibility is required for deep structure
SQL Database	Item metadata, schemas, tables, views, columns, access, lineage and jobs	Read-only fixed catalog queries only
Mirrored Database	Item metadata, provider, selected source context, replication state, SQL tables, views and columns	Source rows, source database names and replication contents are not read
SQL endpoint	Item identity, read-only catalog and storage-to-model bridge	The generated endpoint remains distinct from its parent store
Semantic Model	Tables, columns, measures, descriptions, hidden flags, DAX and object dependencies	Ambiguous references are discarded
Report	Item metadata, bound model, documented owner and pages	PBIR-Legacy pages use compatibility; visual field bindings are not claimed
Notebook	Item metadata, lineage, access and recent runs	Source code and cells are not read
Data Pipeline	Item metadata, lineage, access and recent runs	Activities and expressions are not copied
Dataflow and Datamart	Item metadata, owner and official upstream IDs	Query content is not copied
Eventhouse and KQL Database	Tables, columns, functions, parameters, materialized views, lineage, access and jobs	Business rows, query text and policy bodies are not copied
Ontology	Entities, properties, relationships, bindings and contextualizations	Document and resource-link payloads are excluded
Graph Model	Node types, edge types, properties and source mappings	Graph instances and filter values are excluded

Fabric element	Current depth	Important boundary
Data Agent	Draft and published sources plus selected elements	Instructions, few-shots and prompts are excluded
Fabric App, AppBackend and User Data Function	Top-level identity and exposed relationships	Internal service inventory and function source are not copied
OneLake shortcut	Name, path, target type and bounded target identity	Target URLs, buckets, subpaths and target columns are not stored

16 Read-only Atlas MCP

Version 2 includes an optional local stdio MCP server for deterministic, read-only access to validated Atlas evidence. It runs on the MCP client machine, not as a Rayfin Function or permanent Fabric backend service.

Available tools cover:

- catalog search;
- item context;
- item and object impact;
- access evidence;
- incident evidence;
- snapshot comparison;
- governance findings with priority;
- validated snapshot export.

The server is disabled by default. Activation requires a dedicated public client, delegated `Item.Execute.All`, a reviewed external Entra exchange configuration and live validation under the target Conditional Access policy. No MCP tool changes Fabric permissions or Atlas data.

17 Open source implementation

Fabric Atlas is MIT licensed. The About page exposes the product version, build identifier, source repository, release history and clone command.

Repository area	Responsibility
`src/App.tsx`	Application shell, active workspace and hash navigation
`src/atlas/model.ts`	Shared UI data model
`src/atlas/store.tsx`	Hydration, workspace switching, synchronization and notes
`src/atlas/browser-collector-sync.ts`	Browser-serialized Rayfin collection and compatibility planning
`src/atlas/live-sync.ts`	Snapshot contracts, compatibility UDF invocation and merge logic

Repository area	Responsibility
`src/atlas/scout.ts`	Atlas Scout client for the published Fabric Data Agent
`src/atlas/openlineage.ts`	OpenLineage import, export and external-lineage linking
`src/atlas/sync-brief.ts`	Latest-sync summary and Markdown export
`src/atlas/item-passport.ts`	Item identity sheet export and the Ask Scout item question
`src/atlas/backend.ts`	Workspace-scoped persistence and trusted snapshot loading
`src/atlas/lineage.ts`	Lineage normalization, traversal and layout
`src/atlas/item-relations-evidence.ts`	Preview contracts and relation semantics
`src/atlas/source-provenance-snapshot.ts`	Shortcut, mirroring and materialized lake view projection
`src/mcp/` and `src/atlas/mcp/`	Local read-only MCP transport and tools
`rayfin/functions/`	Typed metadata collectors and durable-execution probes
`rayfin/data/`	Persisted entities and row policies
`fabric/udf/atlas_sync_functions/`	Minimal Python compatibility collector and rollback
`fabric/data-agent/atlas-scout/`	Atlas Scout agent definition and provisioning script

The repository validates release changes with:

```
npm run typecheck
npm test
npm run lint
npm run build
```

18 Current limits

Area	Current state	Required next contract
Scheduled synchronization	Disabled	Supported unattended trigger and identity for every required API
Durable server execution	Additive commands and checkpoints exist as a fail-closed probe	Distributed claim or transaction primitive plus browser-closure recovery
Python compatibility	Limited to Power BI scanner, PBIR-Legacy pages, Kusto live metadata and rollback	Documented Power BI and Kusto audiences for Rayfin Functions

Area	Current state	Required next contract
Atlas MCP	Implemented and disabled by default	Public client, delegated permission, reviewed exchange and live validation
OneLake security	Data access roles, direct members and RLS/CLS presence for up to 40 Lakehouses	Group expansion and constraint evaluation APIs
Purview DLP and Fabric policies	Purview shown as unavailable; Fabric policies listed with enforcement Unknown	Public read APIs for policy evaluation results
Report visual usage	Not collected	Supported report definition path with sensitivity and permission handling
Shared actions and notifications	Optional Teams alert for new Critical findings; no shared assignments	Reviewed ownership model for shared actions
Atlas Scout scope	Read-only, but every agent user sees every synchronized workspace	Row-level filtering or per-workspace scope in the Atlas SQL database
Power BI posture report (PBIP)	Not implemented; JSON and snapshot export, Sync Brief and Teams card cover sharing	Reviewed report template fed by the published snapshot

19 Conclusion

Fabric Atlas gives a Fabric team one consistent metadata system for selected workspaces. The same validated evidence supports catalog search, lineage, Preview review, DAX X-Ray, access analysis, governance, operations and team context.

Its boundaries are part of the product. Atlas does not copy business rows, source code, prompts or credentials. It does not turn missing metadata into a positive control, and it does not claim report field usage that the current APIs do not expose.

Within those limits, Atlas answers four recurring questions:

1. What exists across the selected workspaces?
2. How are items and internal objects connected?
3. What changed, failed or lacks evidence?
4. Who can reach an item, and which recorded path grants that access?

20 Appendix A: figure index

Figure	Subject
1	Governance Radar Overview
2	Item Catalog
3	Workspace-wide search for `asmdb`
4	Asset Catalog
5	Measure expression and dependency evidence
6	Atlas item lineage
7	Item Relations Preview
8	Lineage Evidence
9	Semantic model X-Ray
10	Object lineage
11	Semantic model impact report
12	Model column DAX consumers
13	Object impact report
14	Governance findings
15	Change Center
16	Metadata coverage
17	Governance posture
18	Policies & AI
19	Atlas Scout home
20	Atlas Scout lineage answer
21	Floating Scout with suggested questions
22	Floating Scout live progress
23	External lineage lane on the Map
24	Access Review matrix
25	Principal-centred review
26	Read-only access What-if
27	Departure pack
28	Jobs & health
29	Multi-workspace scope
30	Synchronization
31	Configuration
32	Team notes
33	Architecture and metadata flow
34	Immutable snapshot lifecycle

21 Appendix B: glossary

Term	Meaning in Fabric Atlas
Active workspace	Workspace whose latest trusted snapshot drives the current page
Atlas snapshot	Stored, validated and manifest-backed workspace evidence
Capability	Evidence that a metadata family was collected, unavailable, unsupported or not applicable
Consumer	Item or object that depends on the selected source
DAX edge	Dependency resolved from one model expression to a synchronized object
Effective recorded access	Strongest permission produced by the recorded workspace and item grant paths
Finding	Reproducible governance check over synchronized evidence
Inferred object hop	Cross-item source connection supported by item lineage and one unique schema match
Item Relations Preview	Beta graph built from raw Fabric Item Relations responses
Governance Radar occurrence	New priority finding or risky change in the latest adjacent snapshot pair
Reachable pair	Principal and item combination with recorded effective access
Trusted writer	Configured immutable Entra subject allowed to publish and prune snapshots
What-if	Local simulation that excludes recorded grant paths without changing Fabric

22 References

- Fabric Atlas source: https://github.com/fredgis/FabricAtlas
- Fabric Atlas releases: https://github.com/fredgis/FabricAtlas/releases
- Rayfin source: https://github.com/microsoft/rayfin
- Microsoft Fabric documentation: https://learn.microsoft.com/fabric/